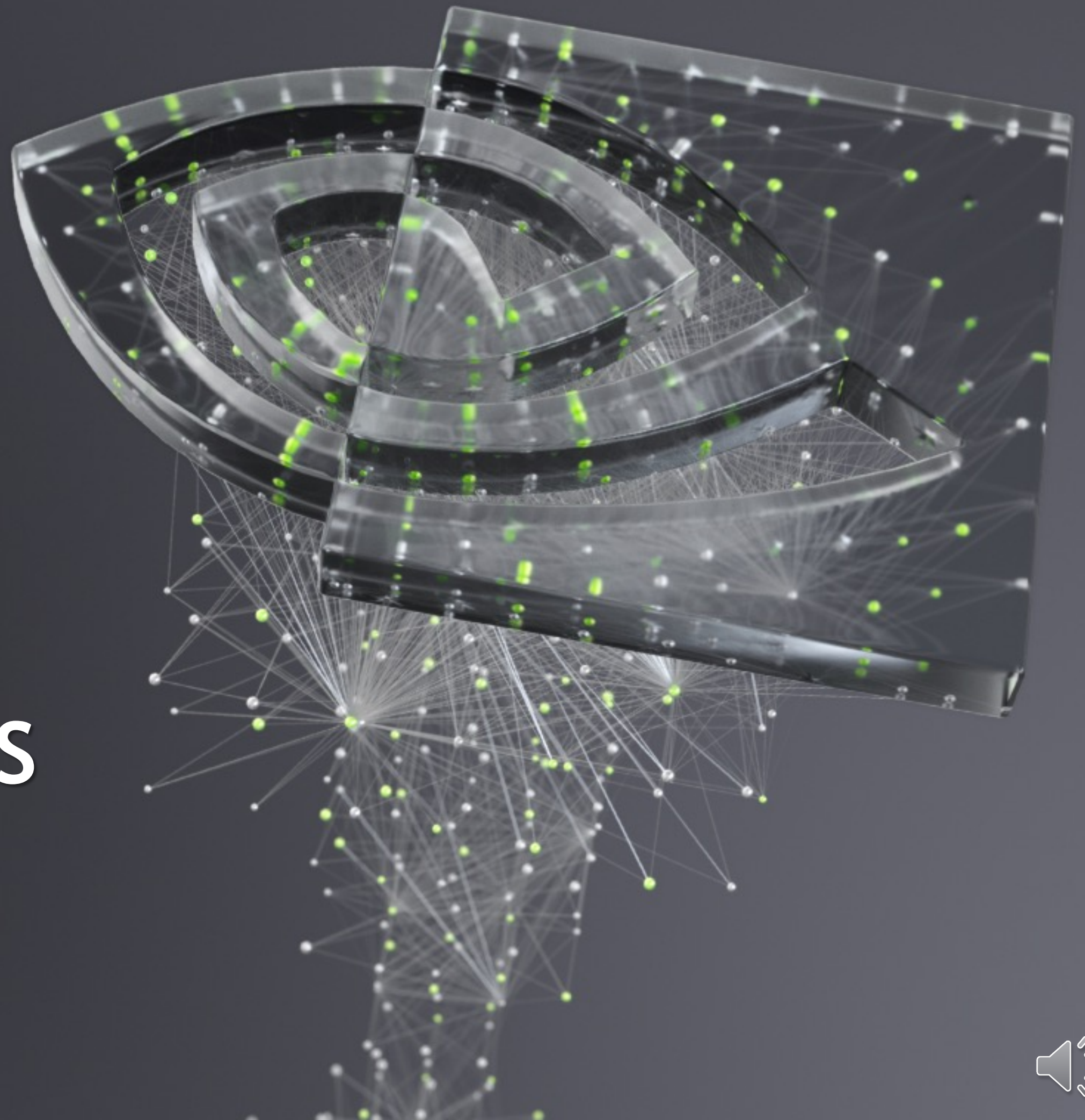




NEARLY EFFORTLESS CUDA GRAPHS

Rob Van der Wijngaart, Jiajie Yao



WE WILL BE STARTING MOMENTARILY

Thank you for joining our talk Nearly Effortless CUDA graphs. Before we start, we'd like to remind you that we are available for chat during the talk. So as questions come up, please click on the 'Chat' button on the bottom right of this web browser to submit your questions

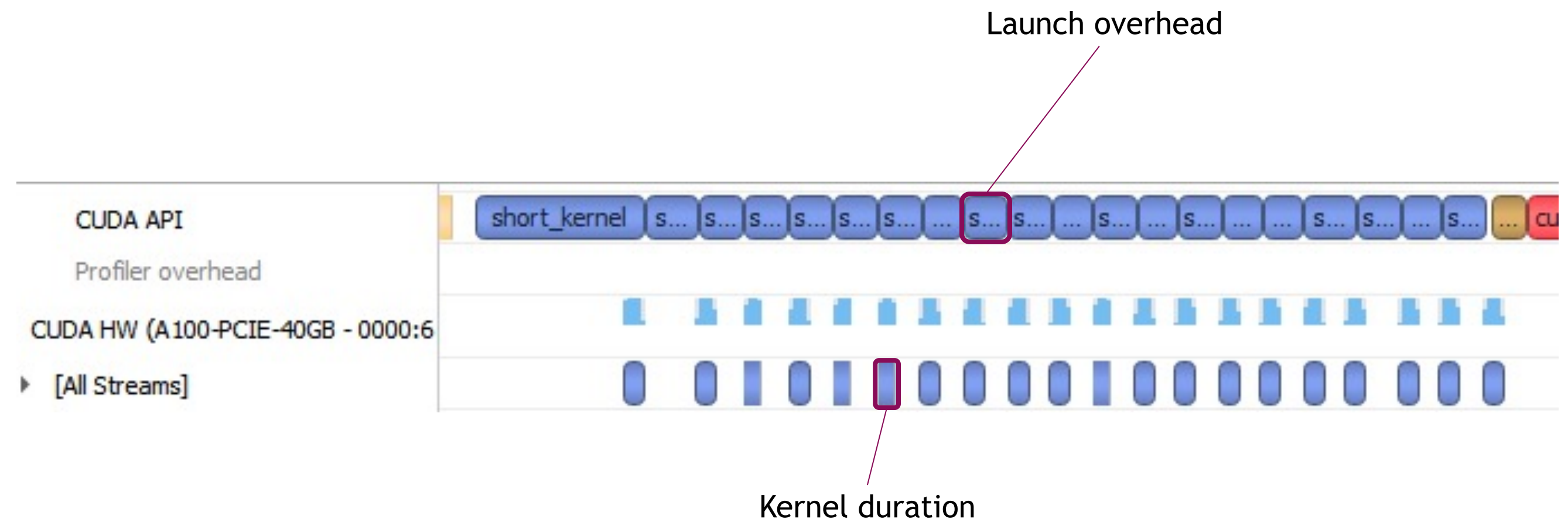


INTRODUCTION TO CUDA GRAPHS

DOCTOR, MY KERNELS HURT

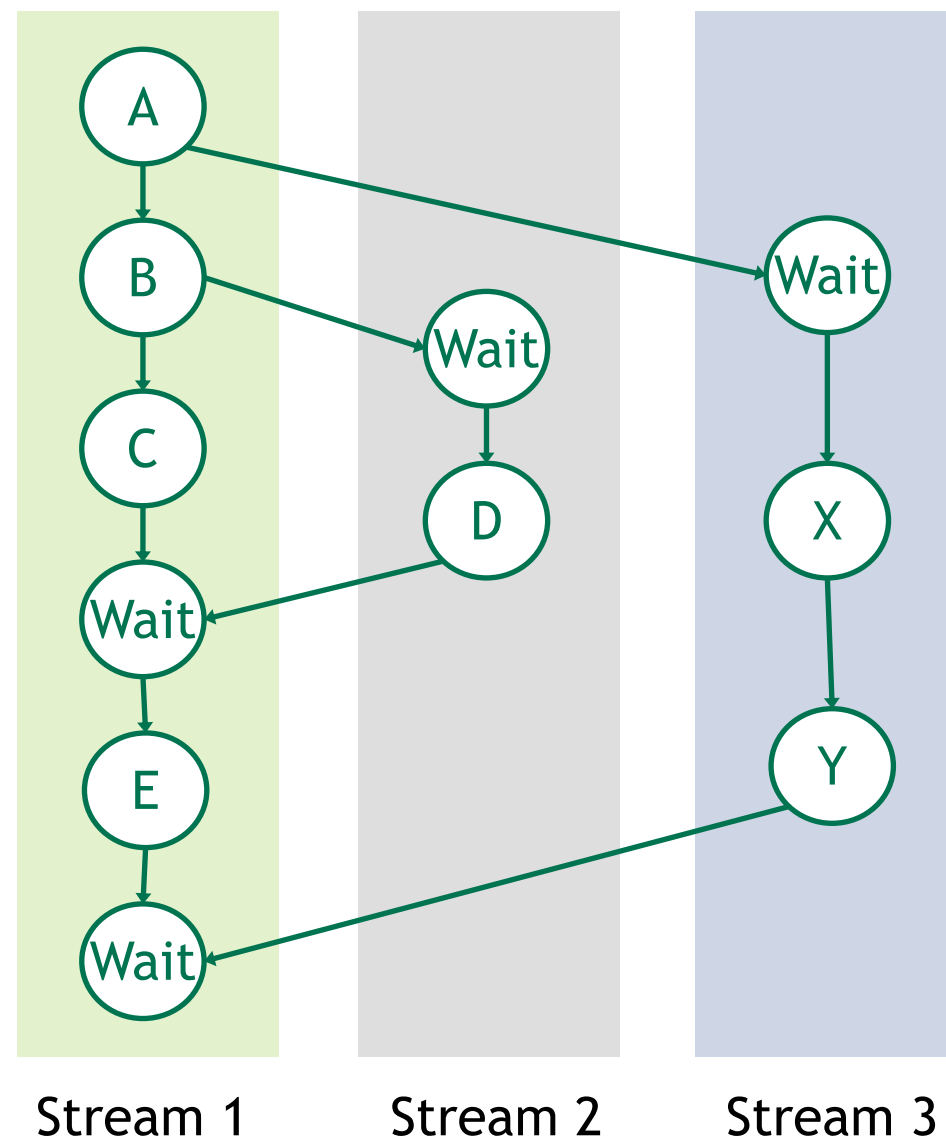
What should I do if my workload has many small kernels

- ▶ Workloads often contain many small kernels—find out using Nsight Systems
- ▶ What is small? It's relative, but typically a few microseconds or less
- ▶ Problem: kernel launch incurs several microseconds overhead on CPU
- ▶ Solution: CUDA Graphs!



ALL CUDA WORK FORMS A GRAPH

CUDA Work in Streams

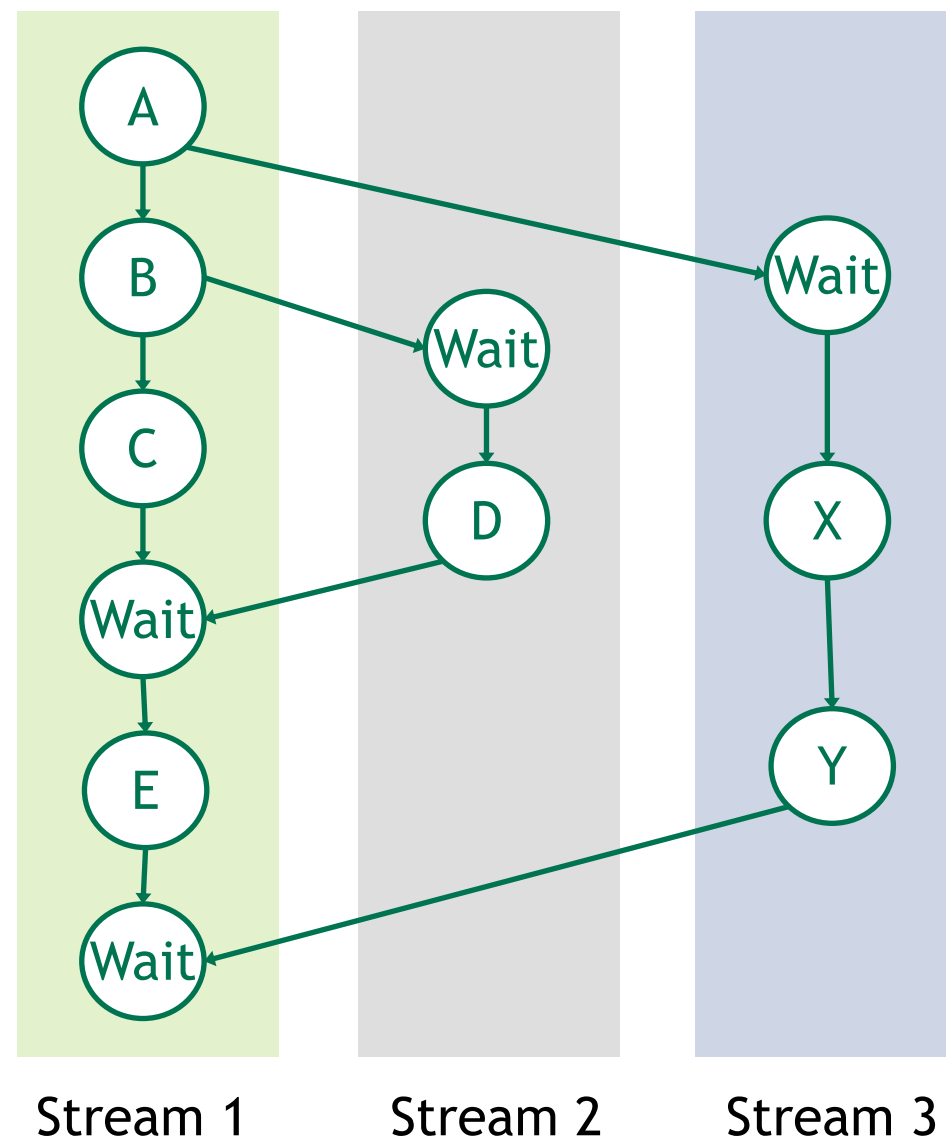


ALL CUDA WORK FORMS A GRAPH

Graph node: any asynchronous CUDA operation

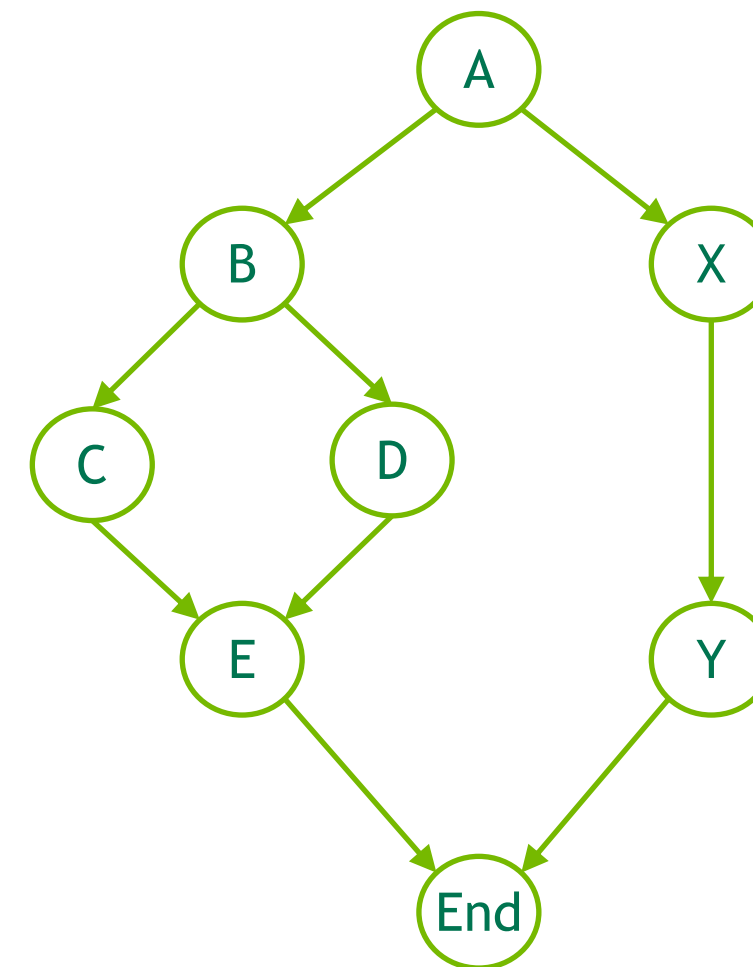
Graph edge: implied or explicit synchronization between nodes

CUDA Work in Streams



Any CUDA
stream
workflow can
be mapped to
a graph

Graph of Dependencies



DEFINITION OF A CUDA GRAPH

Graph node: any asynchronous CUDA operation

Graph edge: implied or explicit synchronization between nodes

Set of operations, connected by dependencies

Operation is (mostly) one of:

Kernel Launch

CUDA kernel running on GPU

CPU Function Call

Callback function on CPU

`cudaMemcpy/MemsetAsync`

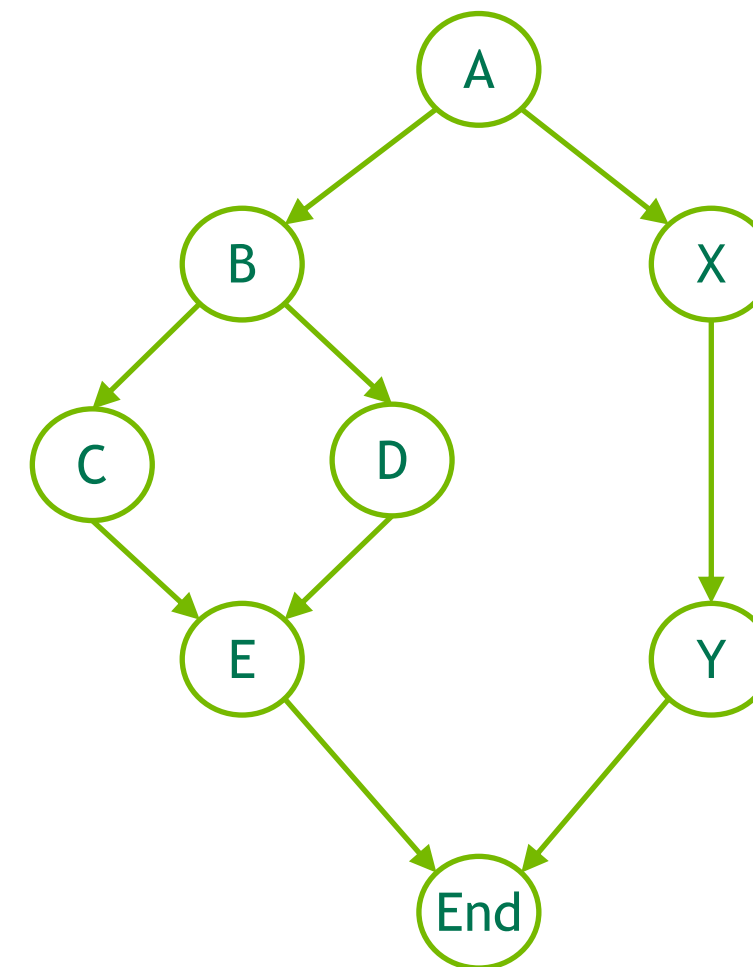
GPU data management

`cudaMalloc/FreeAsync`

GPU memory management

Sub-Graph

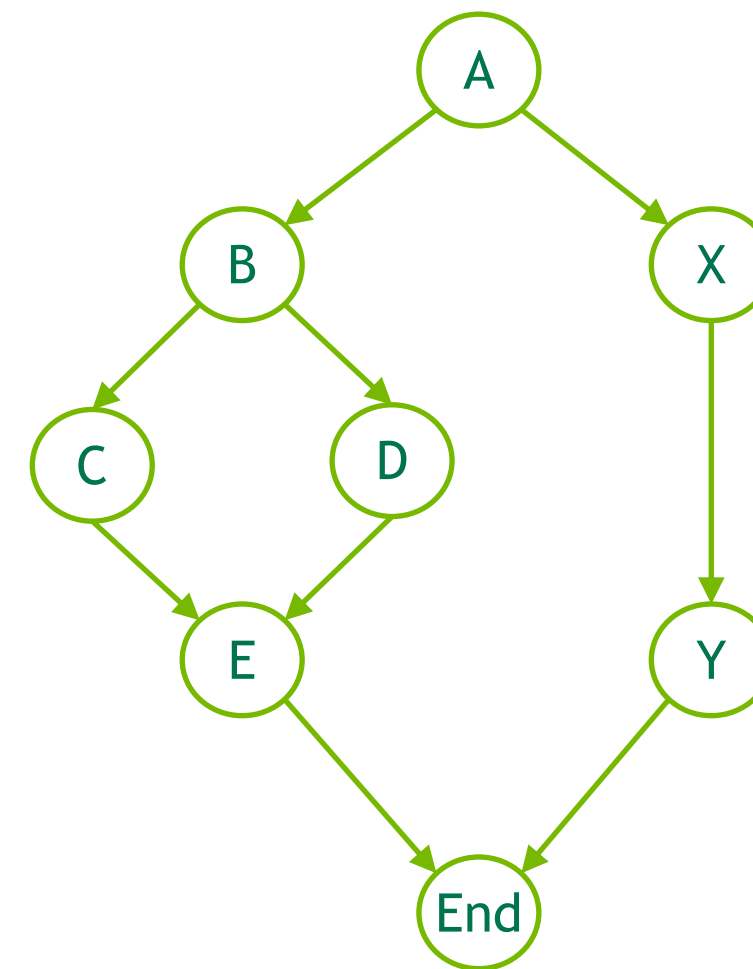
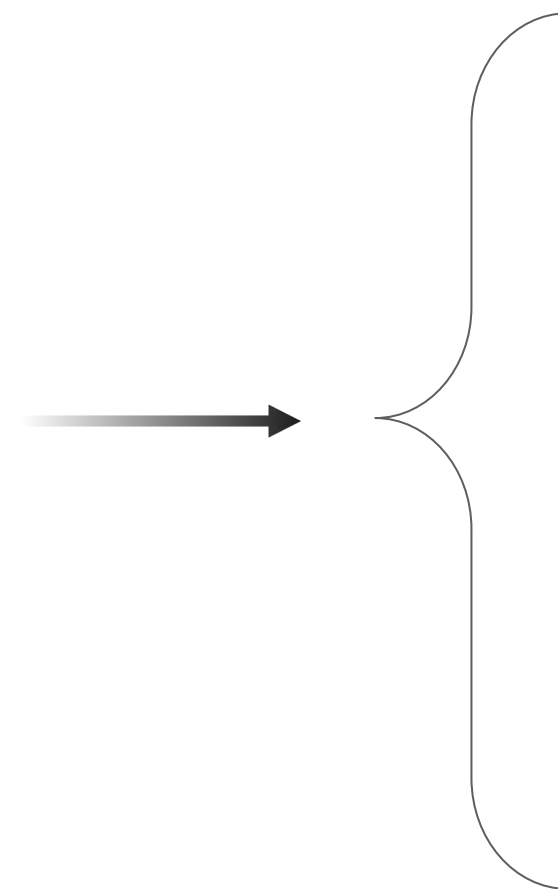
Graphs are hierarchical



CUDA GRAPH EXECUTION MODEL

Graphs generated once, launched repeatedly

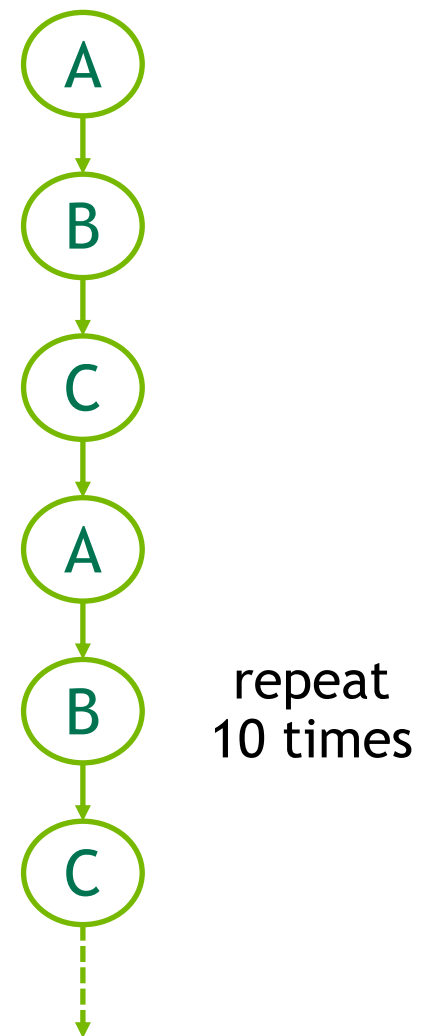
```
for(int i=0; i<1000; i++)  
{  
    launch_graph( G );  
}
```



CUDA GRAPH PROGRAMMING MODEL

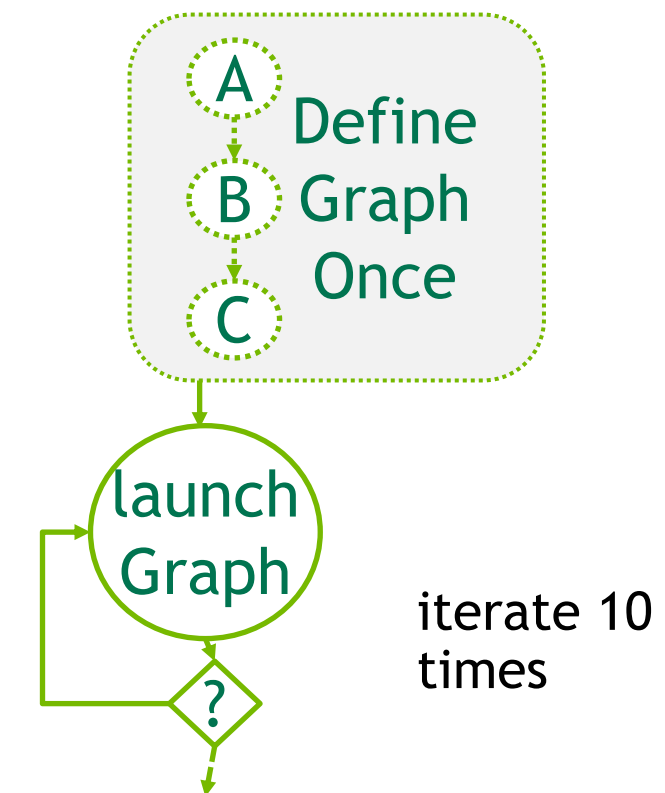
Kernel and Graph execution restrictions

Stream Launch



Parameters: **may** change
Topology: **may** change

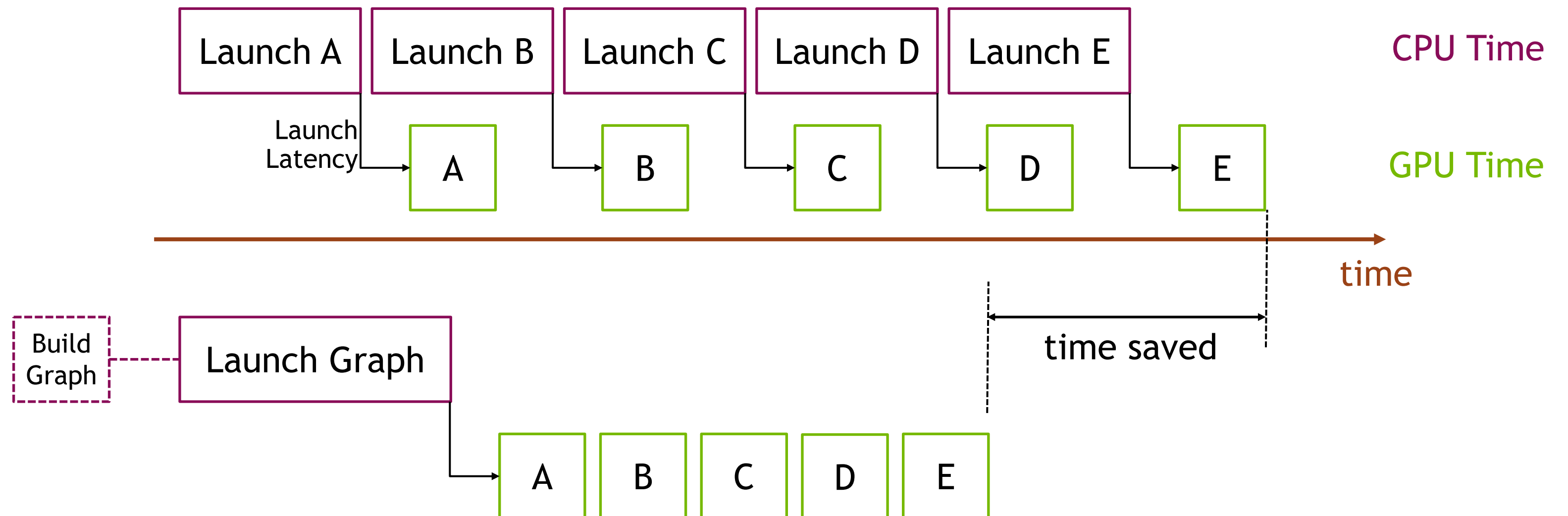
Graph Relaunch



Parameters: **may not** change
Topology: **may not** change

LAUNCH OVERHEAD REDUCTION

Graph launch submits all work at once, reducing CPU cost

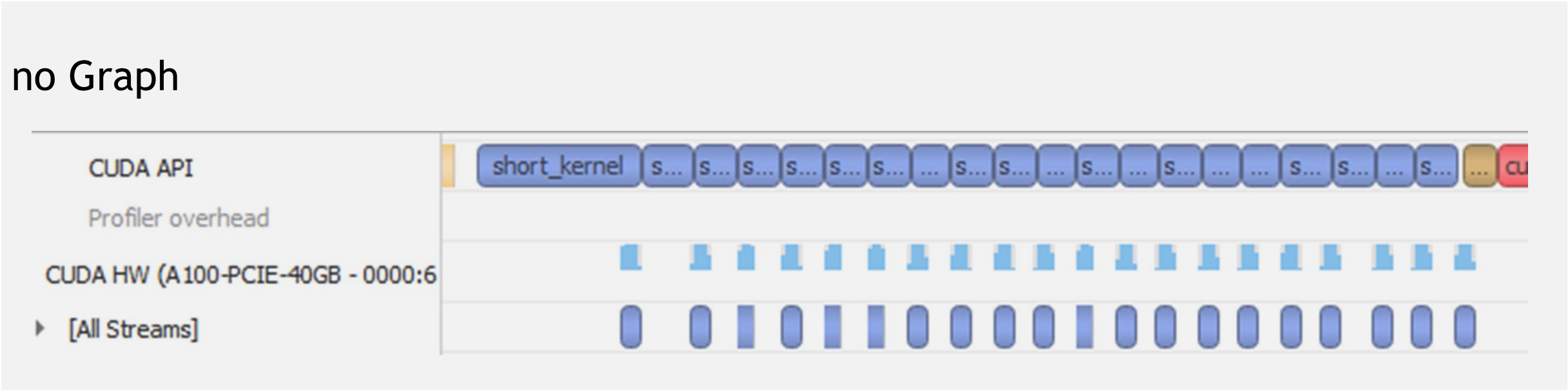


When kernel runtime is short, execution time dominated by CPU launch cost

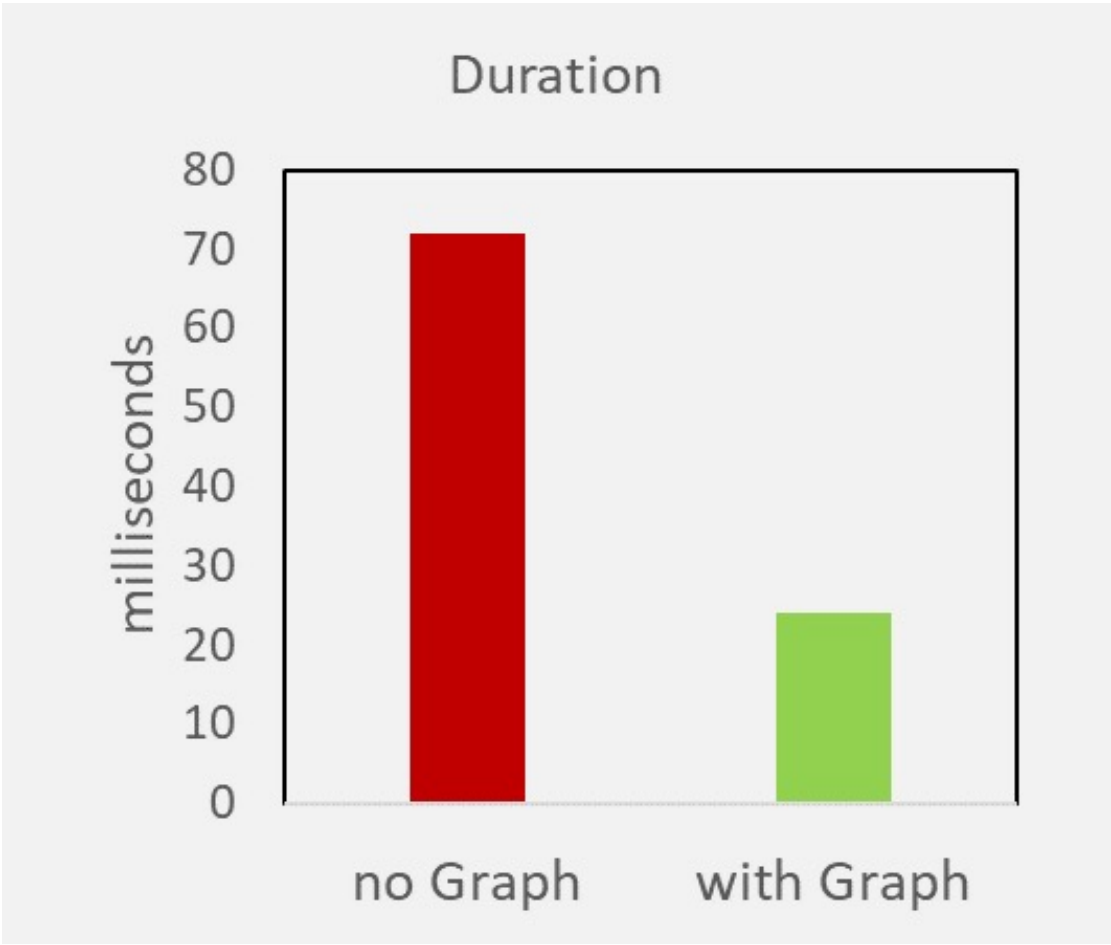
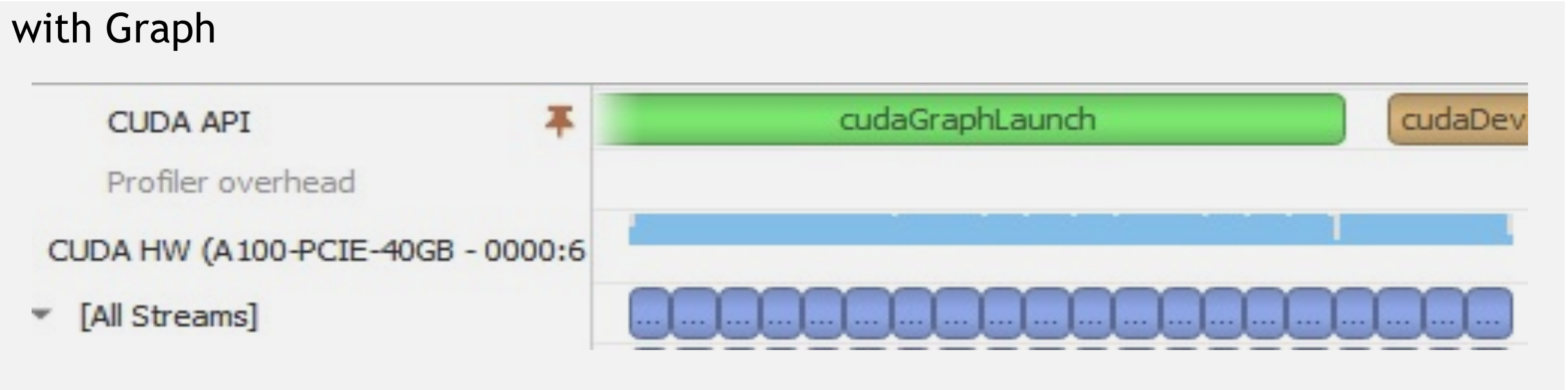
INCREASE THROUGHPUT USING GRAPHS

Kernels themselves also speed up a bit inside Graphs

no Graph



with Graph

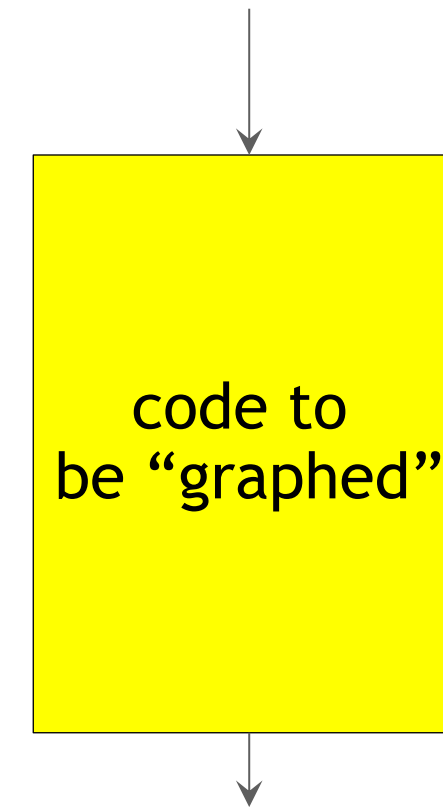


WAYS TO CREATE CUDA GRAPHS

Explicit construction

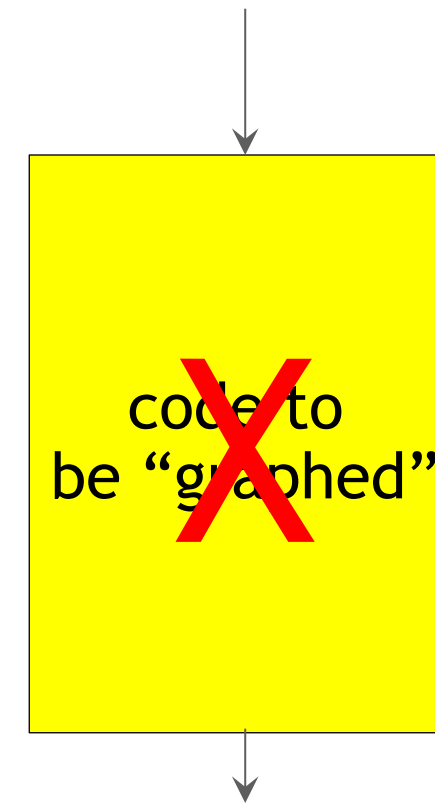
WAYS TO CREATE CUDA GRAPHS

Explicit construction



WAYS TO CREATE CUDA GRAPHS

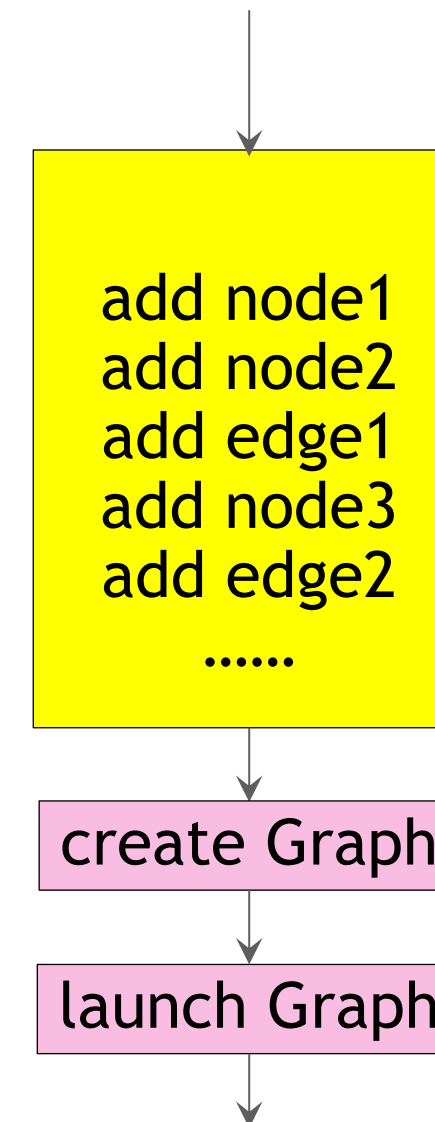
Explicit construction



WAYS TO CREATE CUDA GRAPHS

Explicit construction, using CUDA Graph API calls:

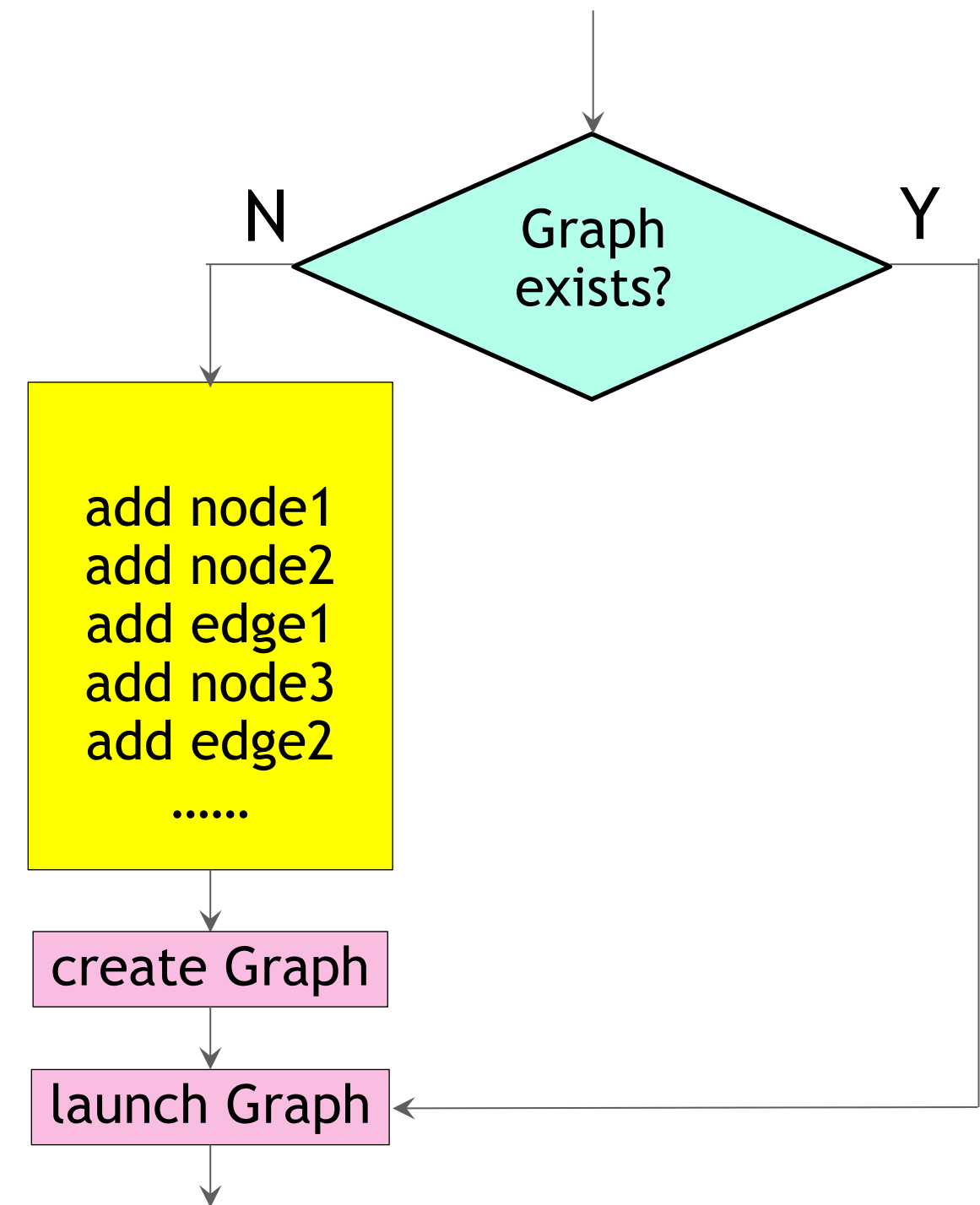
- ▶ Define Graph
- ▶ Add nodes
- ▶ Add dependencies between nodes, representing synchronizations



WAYS TO CREATE CUDA GRAPHS

Explicit construction, using CUDA Graph API calls:

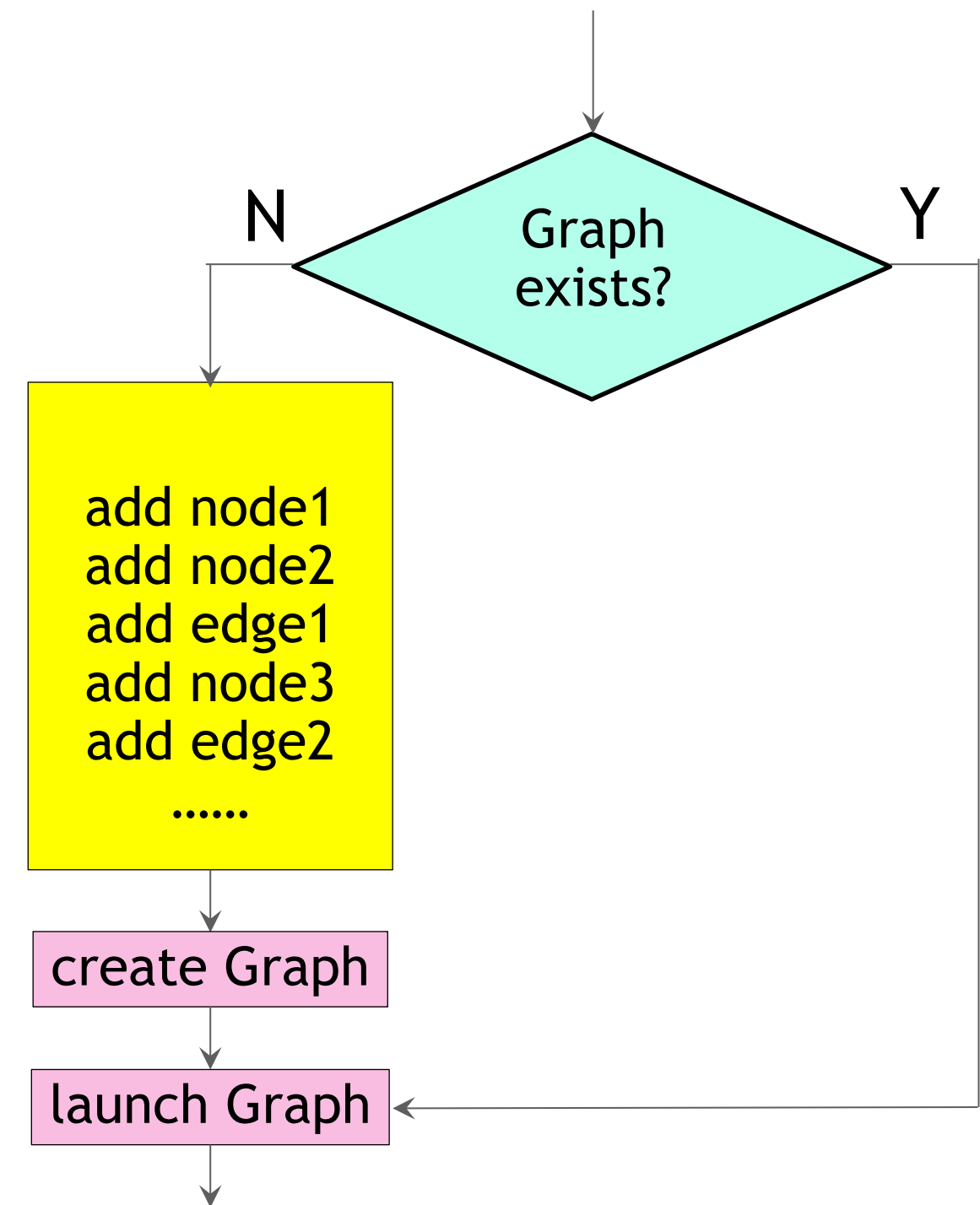
- ▶ Define Graph
- ▶ Add nodes
- ▶ Add dependencies between nodes, representing synchronizations



WAYS TO CREATE CUDA GRAPHS

Explicit construction, using CUDA Graph API calls:

- ▶ Define Graph
- ▶ Add nodes
- ▶ Add dependencies between nodes, representing synchronizations
- ▶ Pros: maximum control
- ▶ Cons: requires effort/knowledge, some code restructuring

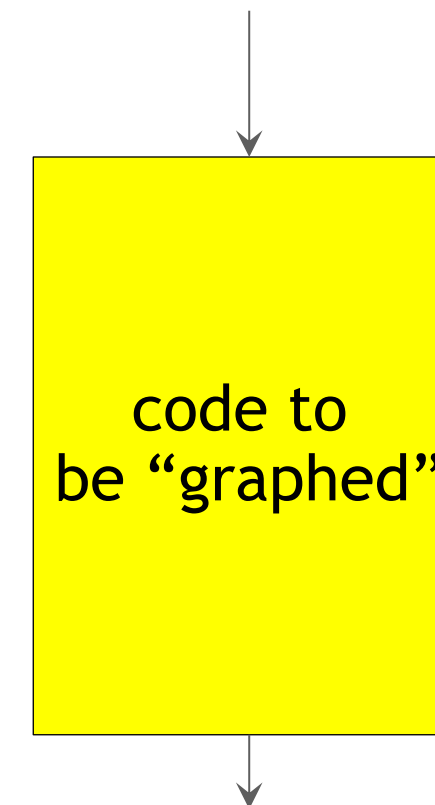


WAYS TO CREATE CUDA GRAPHS

Explicit construction, using CUDA Graph API calls:

- ▶ Define Graph
- ▶ Add nodes
- ▶ Add dependencies between nodes, representing synchronizations
- ▶ Pros: maximum control
- ▶ Cons: requires effort/knowledge, some code restructuring

Capture work done in streams, using CUDA Graph API calls

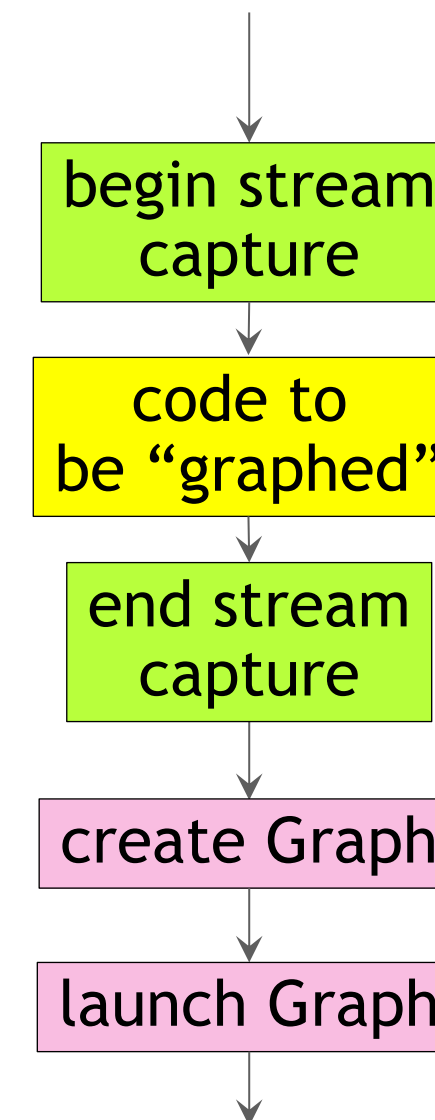


WAYS TO CREATE CUDA GRAPHS

Explicit construction, using CUDA Graph API calls:

- ▶ Define Graph
- ▶ Add nodes
- ▶ Add dependencies between nodes, representing synchronizations
- ▶ Pros: maximum control
- ▶ Cons: requires effort/knowledge, some code restructuring

Capture work done in streams, using CUDA Graph API calls

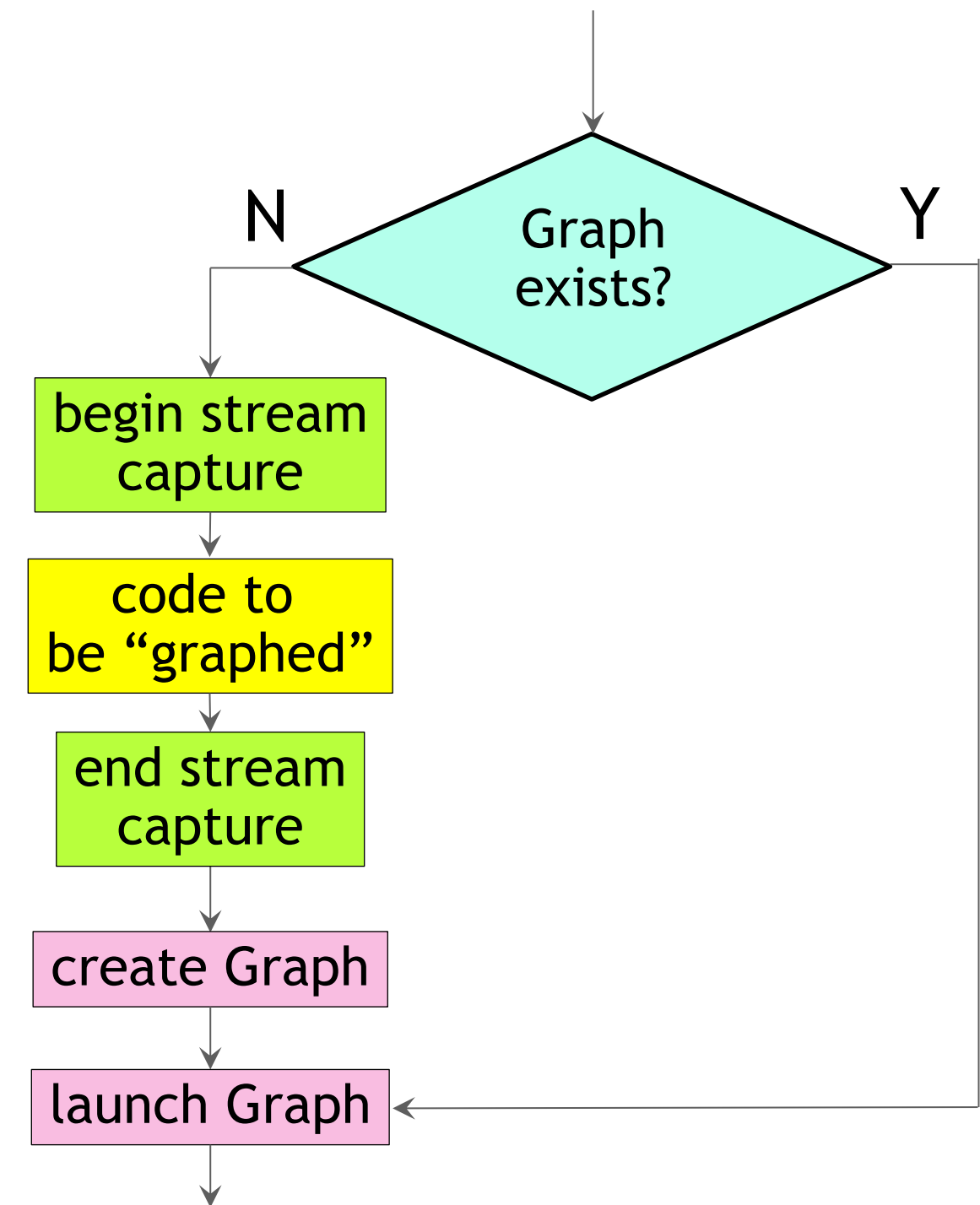


WAYS TO CREATE CUDA GRAPHS

Explicit construction, using CUDA Graph API calls:

- ▶ Define Graph
- ▶ Add nodes
- ▶ Add dependencies between nodes, representing synchronizations
- ▶ Pros: maximum control
- ▶ Cons: requires effort/knowledge, some code restructuring

Capture work done in streams, using CUDA Graph API calls



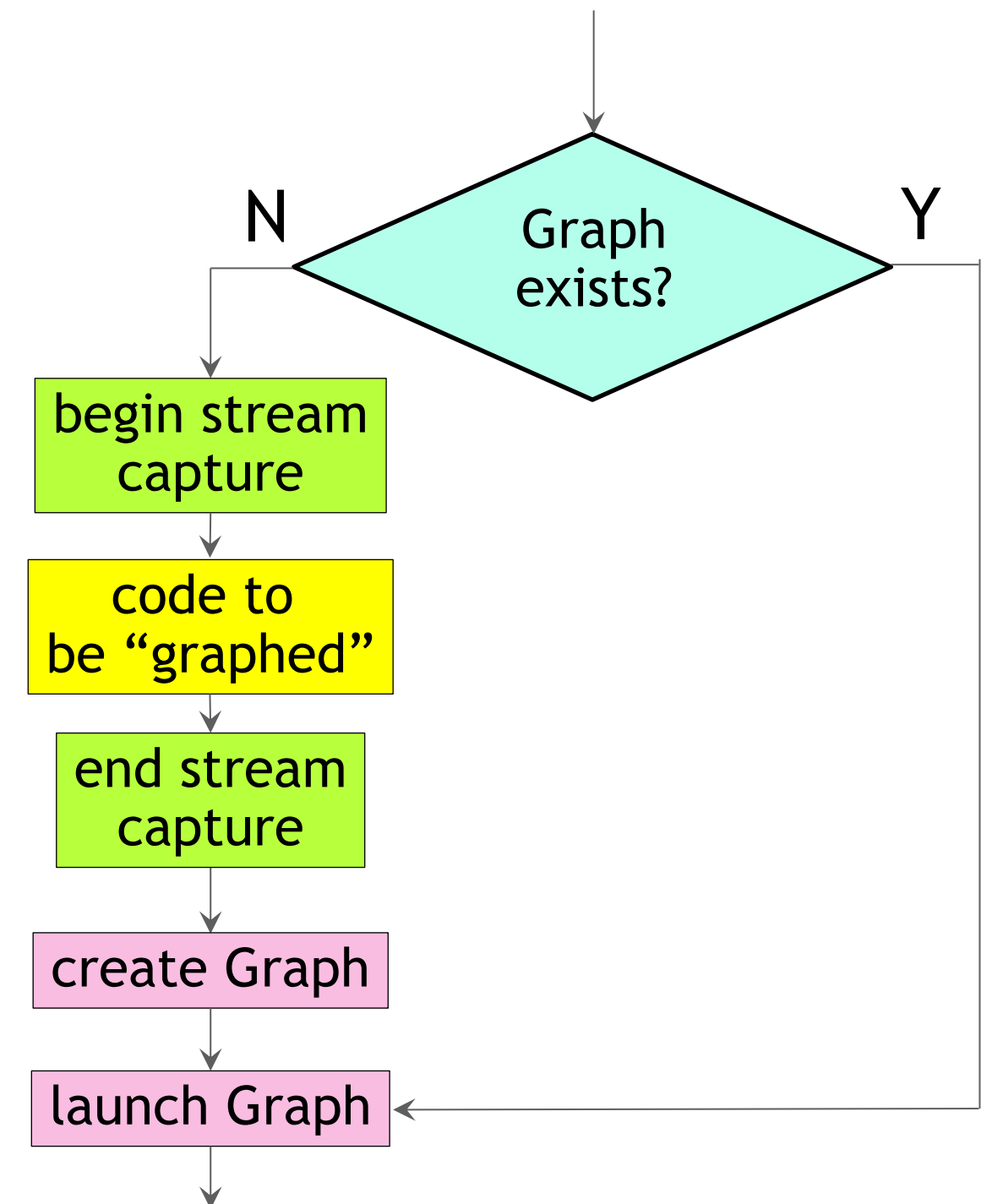
WAYS TO CREATE CUDA GRAPHS

Explicit construction, using CUDA Graph API calls:

- ▶ Define Graph
- ▶ Add nodes
- ▶ Add dependencies between nodes, representing synchronizations
- ▶ Pros: maximum control
- ▶ Cons: requires effort/knowledge, some code restructuring

Capture work done in streams, using CUDA Graph API calls

- ▶ Pros: convenient, done in situ
- ▶ Cons: somewhat less control



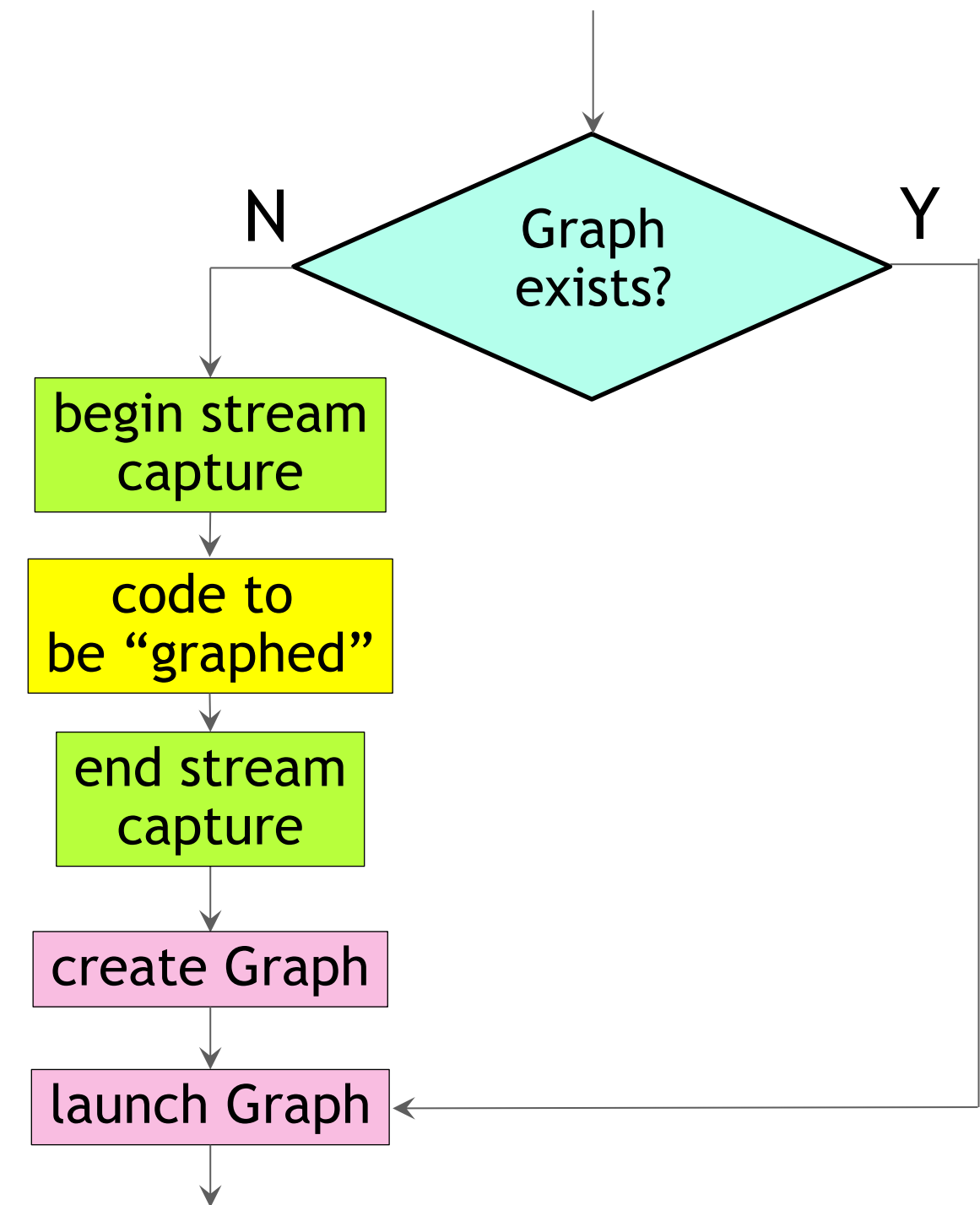
WAYS TO CREATE CUDA GRAPHS

Explicit construction, using CUDA Graph API calls:

- ▶ Define Graph
- ▶ Add nodes
- ▶ Add dependencies between nodes, representing synchronizations
- ▶ Pros: maximum control
- ▶ Cons: requires effort/knowledge, some code restructuring

Capture work done in streams, using CUDA Graph API calls

- ▶ Pros: convenient, done in situ
- ▶ Cons: somewhat less control



CASE STUDY 1: FINANCIAL APPLICATION

REAL-LIFE EXAMPLE

Function in financial application, called multiple times

```
tight_loop(); //function containing many small kernels
```

REAL-LIFE EXAMPLE

Function in financial application, called multiple times

```
cudaGraph_t graph;  
cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal);  
tight_loop(); //function containing many small kernels  
cudaStreamEndCapture(stream, &graph);
```

REAL-LIFE EXAMPLE

Function in financial application, called multiple times

```
cudaGraphExec_t instance;
```

```
cudaGraph_t graph;  
cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal);  
tight_loop(); //function containing many small kernels  
cudaStreamEndCapture(stream, &graph);  
cudaGraphInstantiate(&instance, graph, NULL, NULL, 0);
```

REAL-LIFE EXAMPLE

Function in financial application, called multiple times

```
cudaGraphExec_t instance;
```

```
cudaGraph_t graph;  
cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal);  
tight_loop(); //function containing many small kernels  
cudaStreamEndCapture(stream, &graph);  
cudaGraphInstantiate(&instance, graph, NULL, NULL, 0);
```

```
cudaGraphLaunch(instance, stream);
```


REAL-LIFE EXAMPLE

Function in financial application, called multiple times

```
cudaGraphExec_t instance; bool capture = false;
...
if (!capture)
{
    // we haven't captured the graph yet
    cudaGraph_t graph;
    cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal);
    tight_loop(); //function containing many small kernels
    cudaStreamEndCapture(stream, &graph);
    cudaGraphInstantiate(&instance, graph, NULL, NULL, 0);
    capture = true;
}
cudaGraphLaunch(instance, stream);
```

REAL-LIFE EXAMPLE

Function in financial application, called multiple times

```
void tight_loop(int first_step, MyStruct params, int delta, dim3 grid_dim, dim3 block_dim, cudaStream_t stream)
{
    for (int step = first_step; step >= 0; --step, params.size -= delta)
    {
        tiny_kernel1<<<grid_dim, block_dim, 0, stream>>>(params);
        tiny_kernel2<<<grid_dim, block_dim, 0, stream>>>(params);
    }
}
```

REAL-LIFE EXAMPLE

Function in financial application, called multiple times

```
void tight_loop(int first_step, MyStruct params, int delta, dim3 grid_dim, dim3 block_dim, cudaStream_t stream)
{
    for (int step = first_step; step >= 0; --step, params.size -= delta)
    {
        tiny_kernel1<<<grid_dim, block_dim, 0, stream>>>(params);
        tiny_kernel2<<<grid_dim, block_dim, 0, stream>>>(params);
    }
}
```

Great candidate for CUDA Graph, except:

- ▶ **parameters** and **topology** (number of kernels) may change
- ▶ In principle, need to check all
- ▶ Graphs only give benefit if they can be relaunched

REAL-LIFE EXAMPLE

Function in financial application, called multiple times

```
void tight_loop(int first_step, MyStruct params, int delta, dim3 grid_dim, dim3 block_dim, cudaStream_t stream)
{
    for (int step = first_step; step >= 0; --step, params.size -= delta)
    {
        tiny_kernel1<<<grid_dim, block_dim, 0, stream>>>(params);
        tiny_kernel2<<<grid_dim, block_dim, 0, stream>>>(params);
    }
}
```

printf is your friend

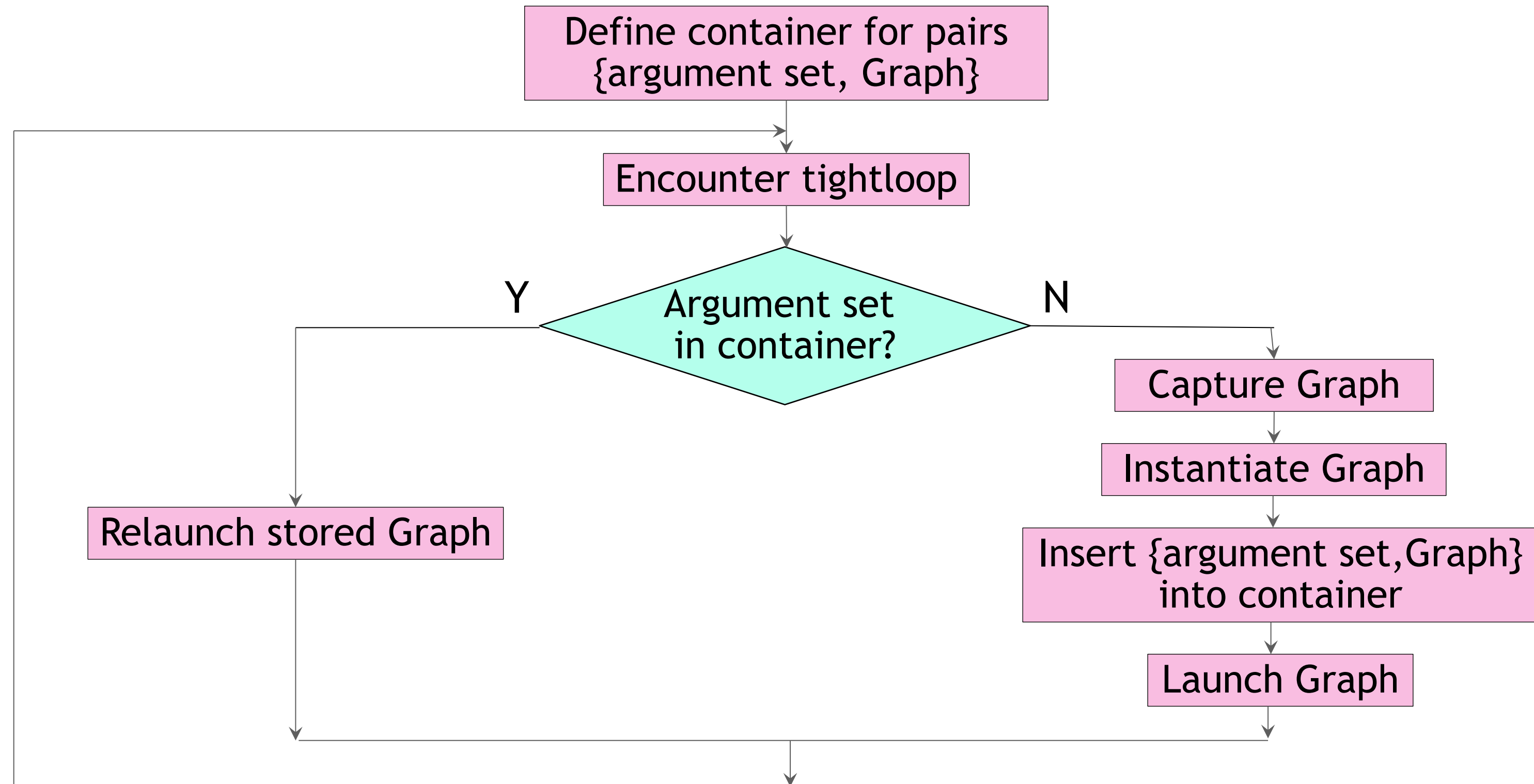
Great candidate for CUDA Graph, except:

- ▶ **parameters** and **topology** (number of kernels) may change
- ▶ In principle, need to check all
- ▶ Graphs only give benefit if they can be relaunched

Workload size	Small	Large
#tightloop calls	70	1540
#iterations	252	1260
Unique argument sets	5	3

SOLUTION 1: RECOGNIZE & SAVE GRAPHS

How are your STL skills?



SOLUTION 1: RECOGNIZE & SAVE GRAPHS

How are your STL skills?

```
map<Marker, cudaGraphExec_t, cmpMarker> MapOfGraphs;
```

```
Marker sextet = {grid_dim, block_dim, first_step, delta, params, stream};
```

```
...
```

```
cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal);
```

```
tight_loop(first_step, params, delta, grid_dim, block_dim, stream);
```

```
cudaStreamEndCapture(stream, &graph);
```

```
cudaGraphInstantiate(&instance, graph, NULL, NULL, 0);
```

```
cudaGraphLaunch(instance, stream);
```

```
}
```

Introduce container
plus elements

Add this if arguments
do change

SOLUTION 1: RECOGNIZE & SAVE GRAPHS

How are your STL skills?

```
map<Marker, cudaGraphExec_t, cmpMarker> MapOfGraphs;
Marker sextet = {grid_dim, block_dim, first_step, delta, params, stream};
...
map<Marker, cudaGraphExec_t, cmpMarker>::iterator it = MapOfGraphs.find(sextet);
if (it == MapOfGraphs.end())
{ // new parameters, so need to capture new graph
    cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal);
    tight_loop(first_step, params, delta, grid_dim, block_dim, stream);
    cudaStreamEndCapture(stream, &graph);
    cudaGraphInstantiate(&instance, graph, NULL, NULL, 0);

    cudaGraphLaunch(instance, stream);
}
```

Introduce container
plus elements

Look for Graph in
container

Add this if arguments
do change

SOLUTION 1: RECOGNIZE & SAVE GRAPHS

How are your STL skills?

```
map<Marker, cudaGraphExec_t, cmpMarker> MapOfGraphs;
Marker sextet = {grid_dim, block_dim, first_step, delta, params, stream};
...
map<Marker, cudaGraphExec_t, cmpMarker>::iterator it = MapOfGraphs.find(sextet);
if (it == MapOfGraphs.end())
{ // new parameters, so need to capture new graph
    cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal);
    tight_loop(first_step, params, delta, grid_dim, block_dim, stream);
    cudaStreamEndCapture(stream, &graph);
    cudaGraphInstantiate(&instance, graph, NULL, NULL, 0);
    // add this graph to the map of saved graphs
    MapOfGraphs.insert(make_pair(sextet, instance));
    cudaGraphLaunch(instance, stream);
}
```

Introduce container
plus elements

Look for Graph in
container

Add this if arguments
do change

Add Graph to container

SOLUTION 1: RECOGNIZE & SAVE GRAPHS

How are your STL skills?

```
map<Marker, cudaGraphExec_t, cmpMarker> MapOfGraphs;
```

```
Marker sextet = {grid_dim, block_dim, first_step, delta, params, stream};
```

```
...
```

```
map<Marker, cudaGraphExec_t, cmpMarker>::iterator it = MapOfGraphs.find(sextet);
```

```
if (it == MapOfGraphs.end())
```

```
{ // new parameters, so need to capture new graph
```

```
    cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal);
```

```
    tight_loop(first_step, params, delta, grid_dim, block_dim, stream);
```

```
    cudaStreamEndCapture(stream, &graph);
```

```
    cudaGraphInstantiate(&instance, graph, NULL, NULL, 0);
```

```
    // add this graph to the map of saved graphs
```

```
    MapOfGraphs.insert(make_pair(sextet, instance));
```

```
    cudaGraphLaunch(instance, stream);
```

```
}
```

```
else
```

```
{// relaunch previously captured graph
```

```
    cudaGraphLaunch(it->second, stream);
```

```
}
```

Introduce container
plus elements

Look for Graph in
container

Add this if arguments
do change

Add Graph to container

Launch Graph in
container

SOLUTION 1: RECOGNIZE & SAVE GRAPHS

How are your STL skills?

```
map<Marker, cudaGraphExec_t, cmpMarker> MapOfGraphs;
Marker sextet = {grid_dim, block_dim, first_step, delta, params, stream};
...
map<Marker, cudaGraphExec_t, cmpMarker>::iterator it = MapOfGraphs.find(sextet);
if (it == MapOfGraphs.end())
{ // new parameters, so need to capture new graph
  cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal);
  tight_loop(first_step, params, delta, grid_dim, block_dim, stream);
  cudaStreamEndCapture(stream, &graph);
  cudaGraphInstantiate(&instance, graph, NULL, NULL, 0);
  // add this graph to the map of saved graphs
  MapOfGraphs.insert(make_pair(sextet, instance));
  cudaGraphLaunch(instance, stream);
}
else
{ // relaunch previously captured graph
  cudaGraphLaunch(it->second, stream);
}
```

Introduce container
plus elements

Look for Graph in
container

Add this if arguments
do change

Plus cmpMarker function to
compare different instances
of grid_dim, block_dim,
first_step, delta, stream,
params.X

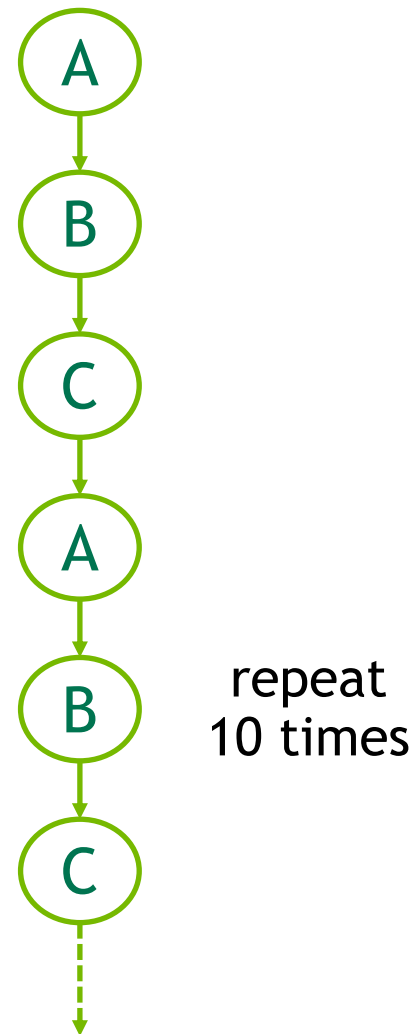
Add Graph to container

Launch Graph in
container

SOLUTION 2: UPDATE GRAPHS

Kernel and Graph execution restrictions

Stream Launch



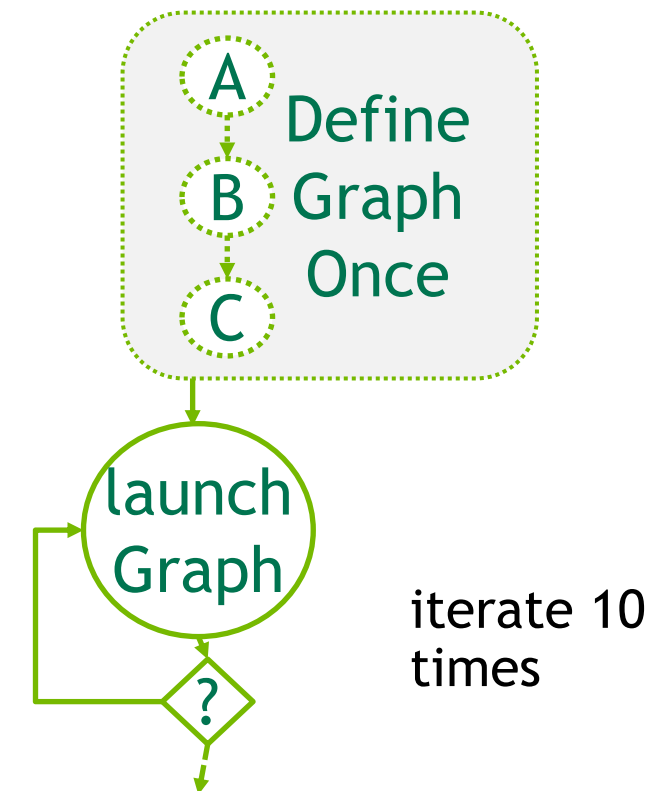
Parameters: **may** change
Topology: **may** change

Graph Update

- Compare existing Graph with newly captured Graph
- If possible, update parameters of existing Graph

Parameters: **may** change
Topology: **may not** change

Graph Relaunch

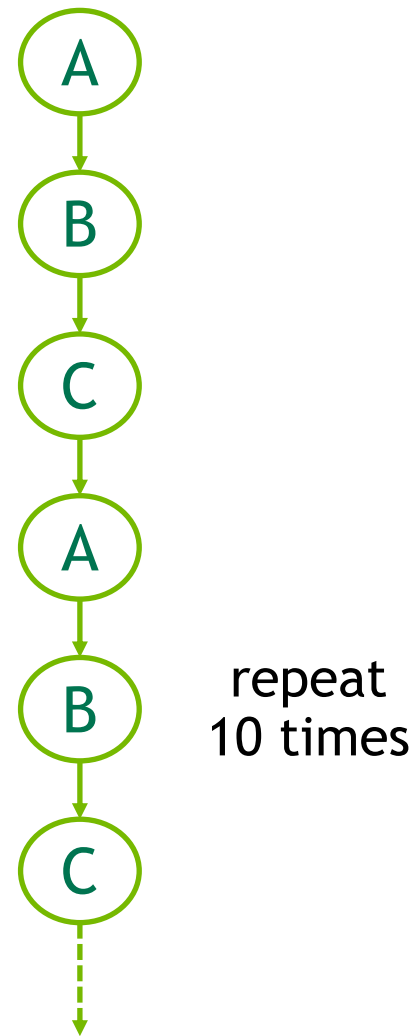


Parameters: **may not** change
Topology: **may not** change

SOLUTION 2: UPDATE GRAPHS

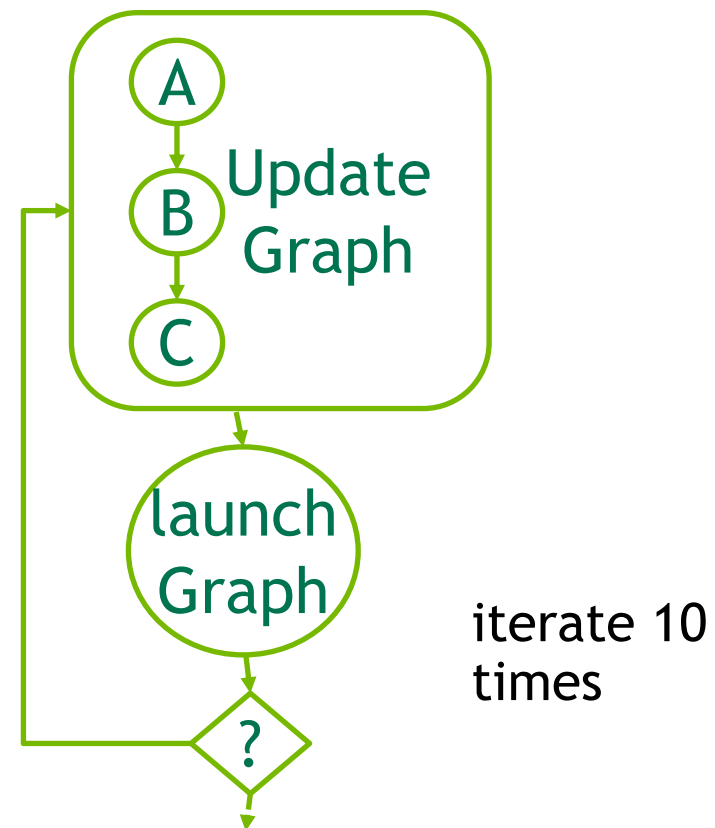
Kernel and Graph execution restrictions

Stream Launch



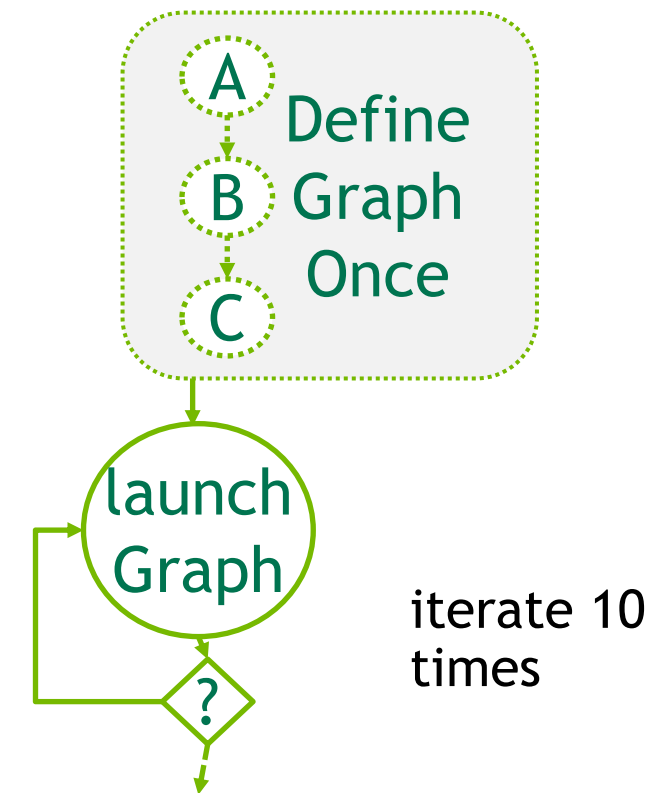
Parameters: **may** change
Topology: **may** change

Graph Update



Parameters: **may** change
Topology: **may not** change

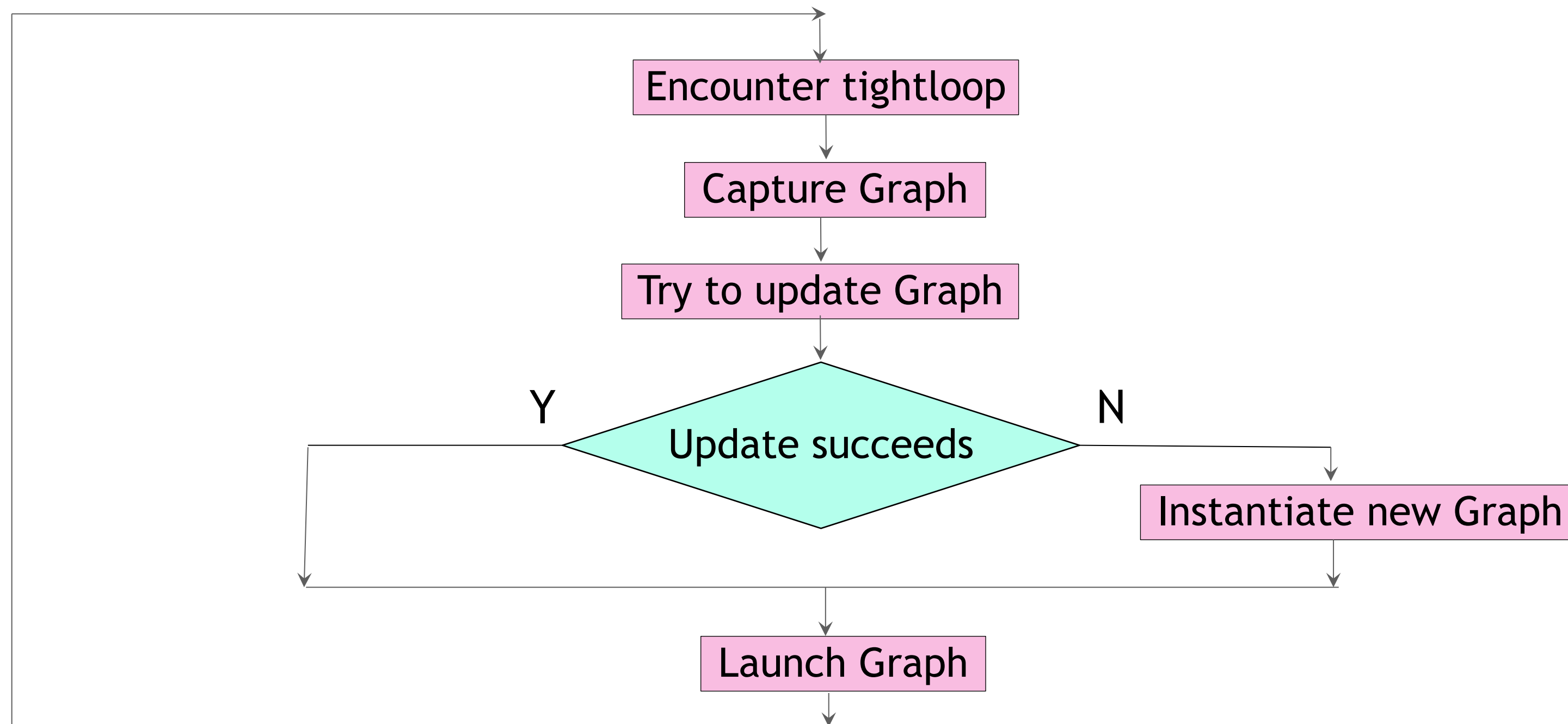
Graph Relaunch



Parameters: **may not** change
Topology: **may not** change

SOLUTION 2: UPDATE GRAPHS

No STL skills required



SOLUTION 2: UPDATE GRAPHS

```
cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal);  
tight_loop(first_step, params, delta, grid_dim, block_dim, stream);  
cudaStreamEndCapture(stream, &graph);
```

ALWAYS capture Graph

SOLUTION 2: UPDATE GRAPHS

```
bool capture= false;
...
cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal);
tight_loop(first_step, params, delta, grid_dim, block_dim, stream);
cudaStreamEndCapture(stream, &graph);

if (!capture) {
    cudaGraphInstantiate(&instance, graph, NULL, NULL, 0);
    capture = true;
}

cudaGraphLaunch(instance, stream);
```

ALWAYS capture Graph

SOLUTION 2: UPDATE GRAPHS

```
bool capture= false;
...
cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal);
tight_loop(first_step, params, delta, grid_dim, block_dim, stream);
cudaStreamEndCapture(stream, &graph);

if (!capture) {
    cudaGraphInstantiate(&instance, graph, NULL, NULL, 0);
    capture = true;
} else {
    // graph already exists; try to apply changes
    if (cudaGraphExecUpdate(instance, graph, NULL, &update)

}

cudaGraphLaunch(instance, stream);
```

ALWAYS capture Graph

Update Graph and go

SOLUTION 2: UPDATE GRAPHS

```
bool capture= false;
...
cudaStreamBeginCapture(stream, cudaStreamCaptureModeGlobal);
tight_loop(first_step, params, delta, grid_dim, block_dim, stream);
cudaStreamEndCapture(stream, &graph);

if (!capture) {
    cudaGraphInstantiate(&instance, graph, NULL, NULL, 0);
    capture = true;
} else {
    // graph already exists; try to apply changes
    if (cudaGraphExecUpdate(instance, graph, NULL, &update) != cudaSuccess)
        cudaGraphInstantiate(&instance, graph, NULL, NULL, 0);
}
cudaGraphLaunch(instance, stream);
```

ALWAYS capture Graph

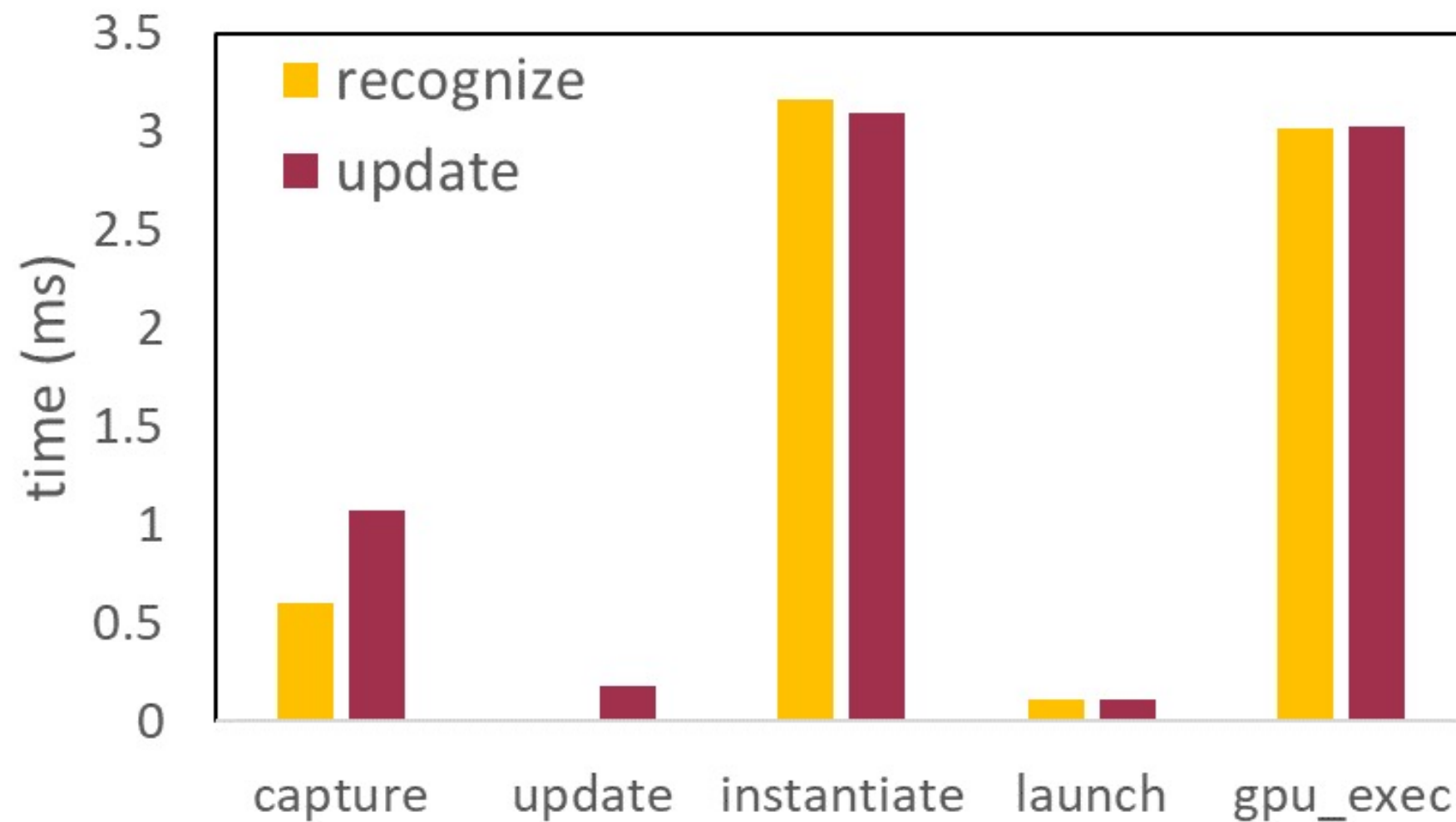
Update Graph and go

Update failed, reinstantiate

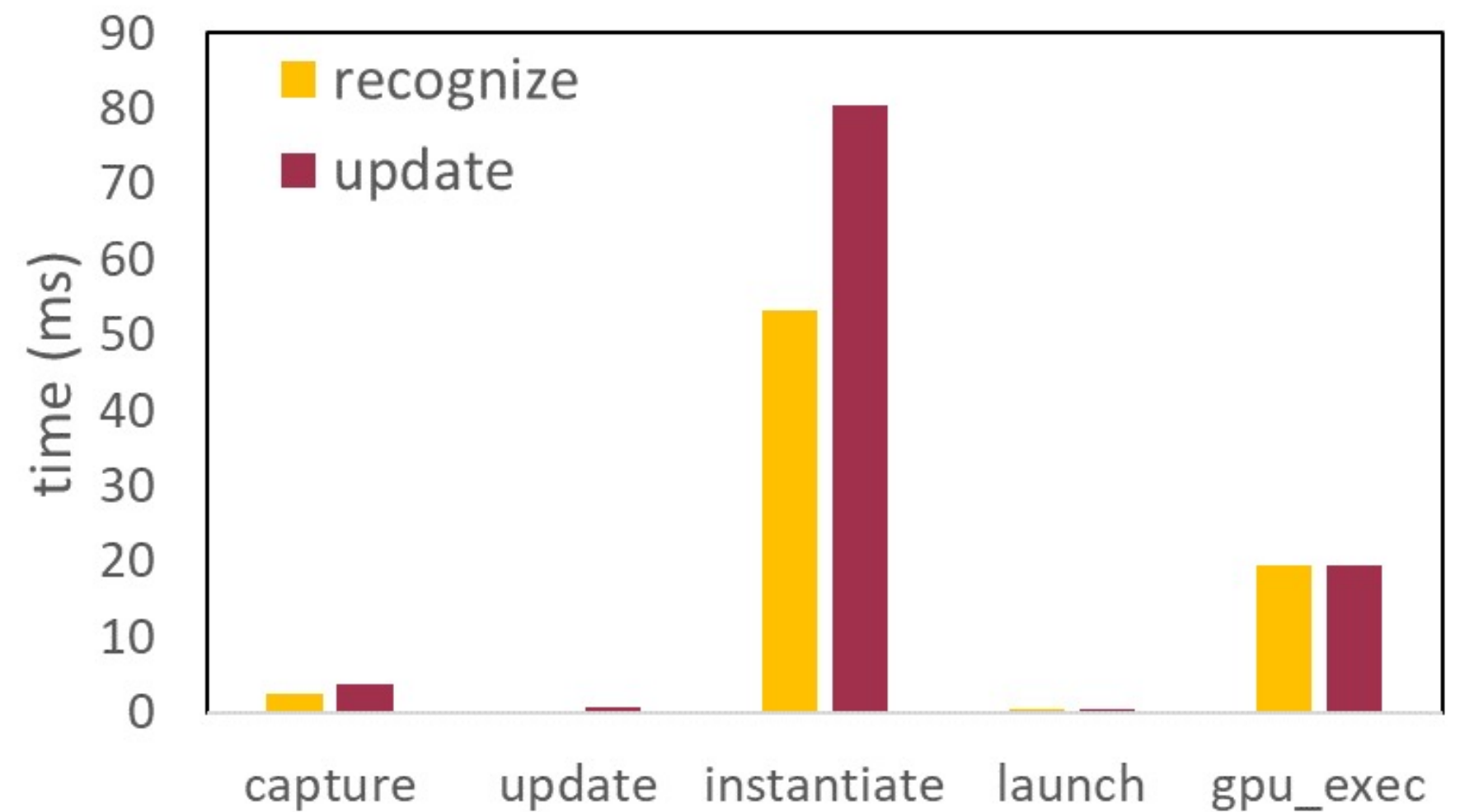
SOLUTION 2: UPDATE GRAPHS

Capture, update, launch cheap compared to instantiate, run

Average time per operation, small workload



Average time per operation, large workload



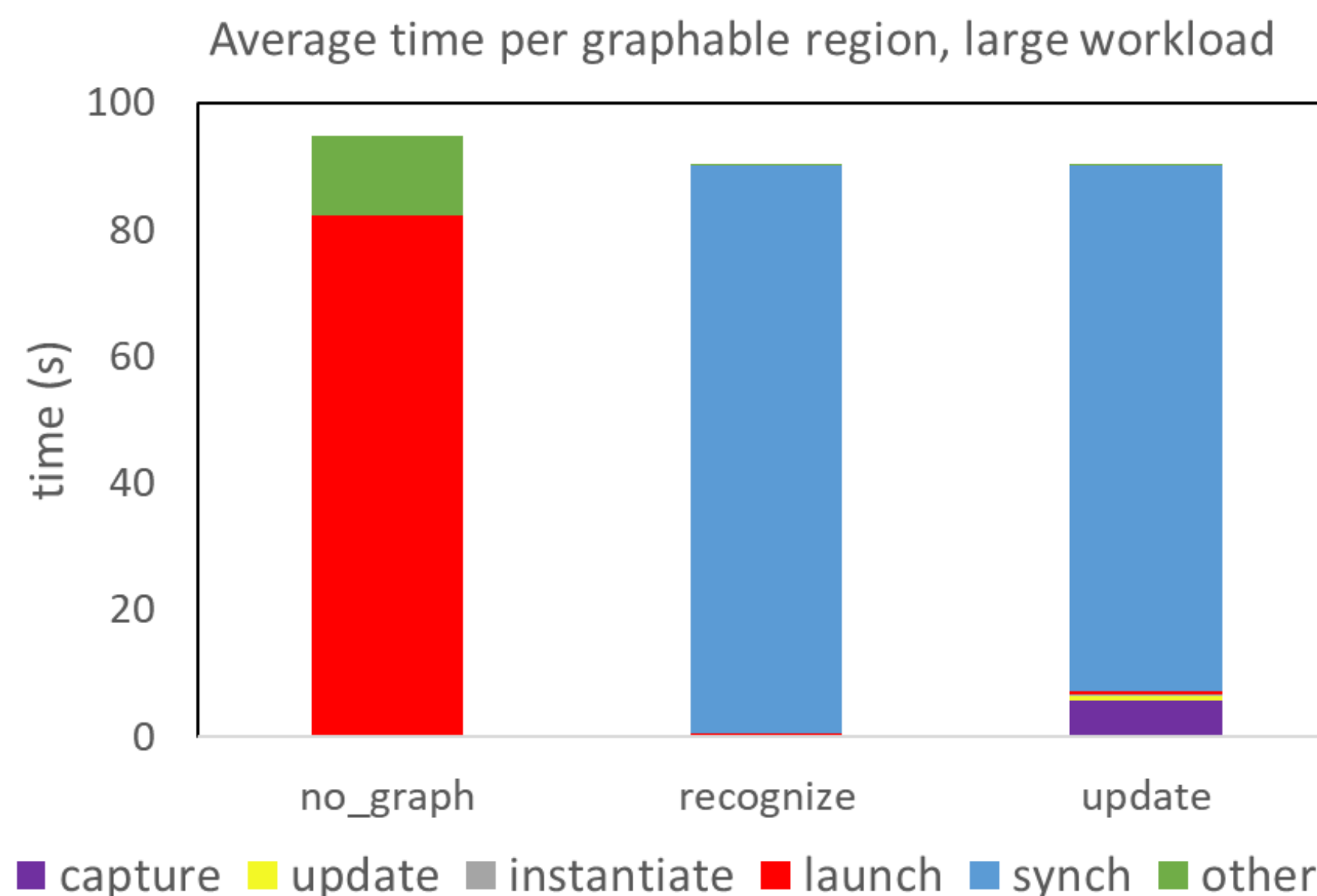
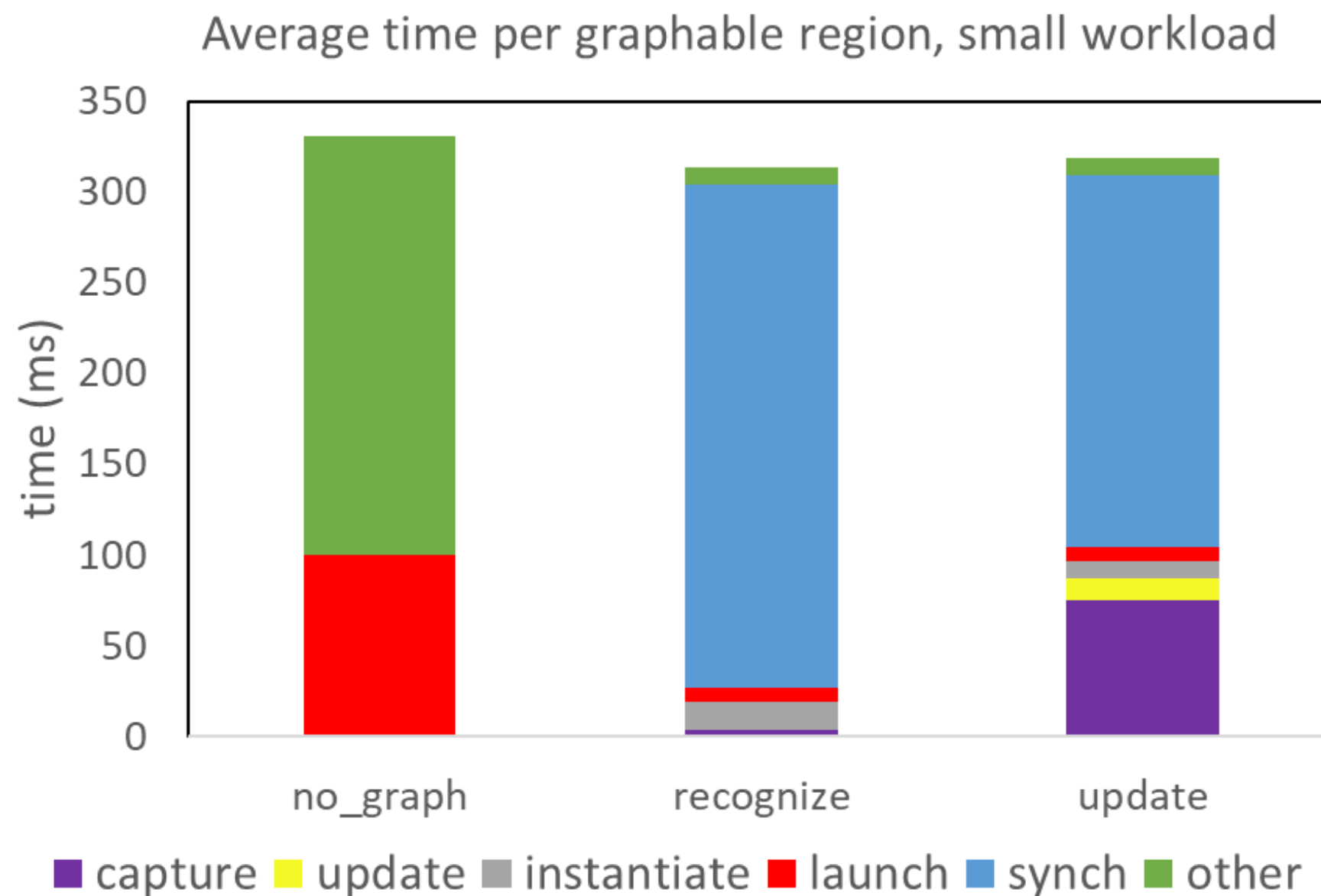
SOLUTION 2: UPDATE GRAPHS

Expensive instantiations are infrequent

Workload size	Small		Large	
Kernels/Graph	504		2520	
Graph launches	70		1540	
Method	update	recognize	update	recognize
Instantiations	3	5	1	3

COMPARING RECOGNIZE AND UPDATE

Launch time reduces greatly with CUDA Graphs



COMPARING RECOGNIZE AND UPDATE

Recognize

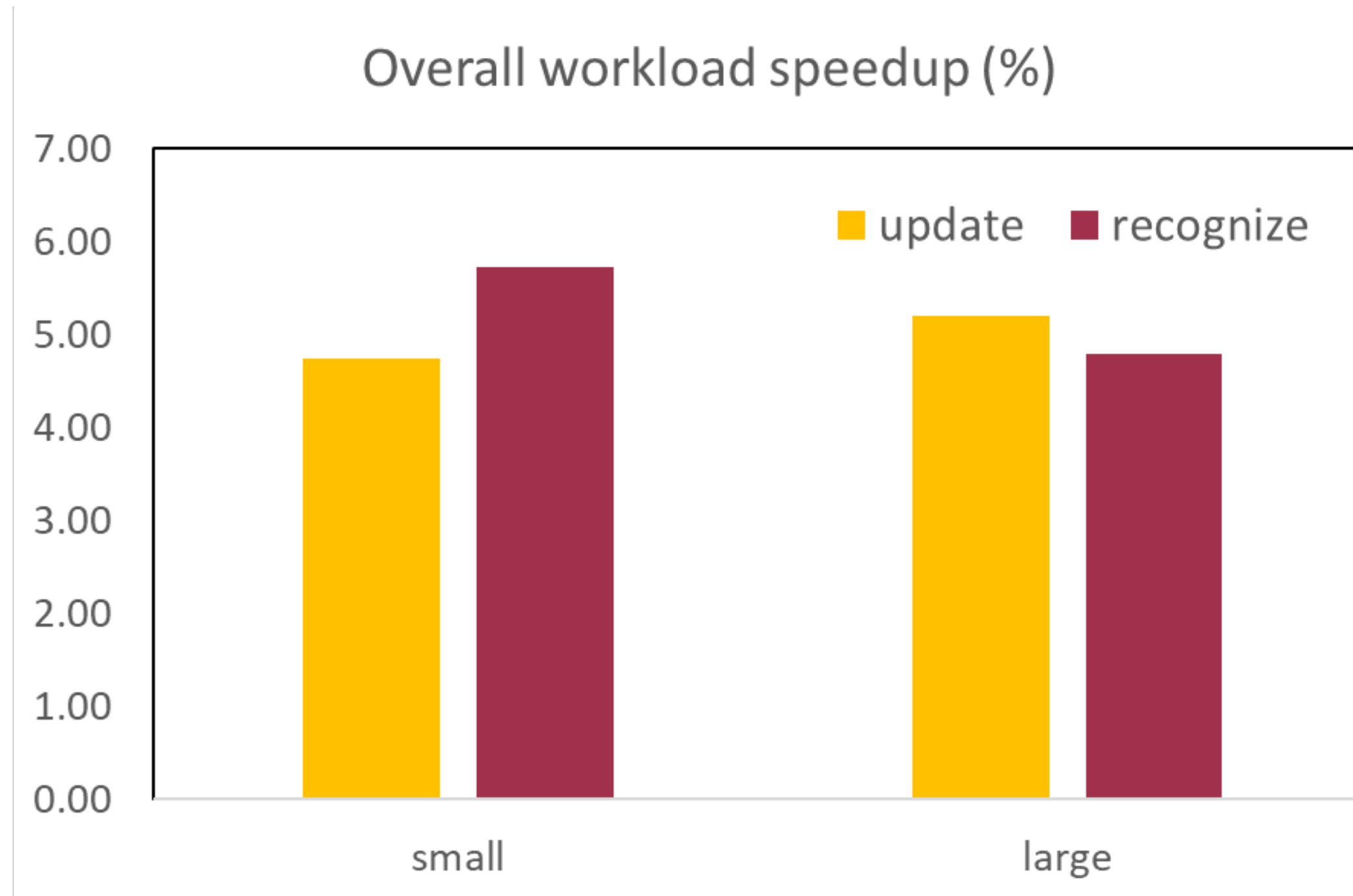
- ▶ Best when small number of unique Graphs, repeated many times
- ▶ Independent of order in which graphs encountered
- ▶ Negligible overhead when saved Graph recognized
- ▶ May have to save many Graphs

Update

- ▶ Best when small number of unique topologies, repeated many times
- ▶ Worst when cycling between Graphs with different topologies
- ▶ Always some stream capture overhead
- ▶ Never need to save more than one Graph

COMPARING RECOGNIZE AND UPDATE

Graphable region less than 30%/20% of overall small/large workload



SUMMARY

CUDA Graphs can give substantial performance benefits with little effort

Criteria for introducing CUDA Graphs:

- ▶ Repeated sets of short GPU tasks (e.g. kernels)
- ▶ Only asynchronous operations

Efficient way to create CUDA Graphs

- ▶ Stream capture

Different approaches for different situations

- ▶ If identical sets of kernels recur: almost no work
- ▶ If topology and/or parameters change frequently: consider saving and recognizing Graphs
- ▶ If topology changes infrequently: consider updating Graphs

CASE STUDY 2: INFERENCE IN RECOMMENDATION SYSTEM

CUDAGRAPH IN TENSORFLOW

What happens in TF session run

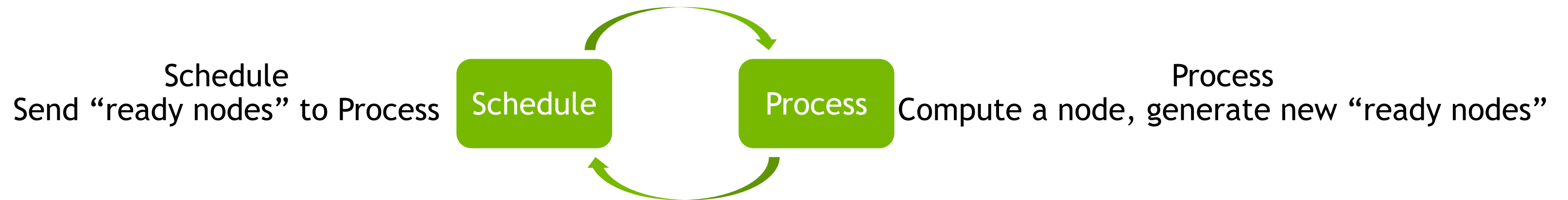
Integrate CUDA Graph into TensorFlow

Use case: Alibaba Search Recommendation System

WHAT HAPPENS IN SESSION.RUN

How Are Operations Scheduled

Schedule - Process Loop

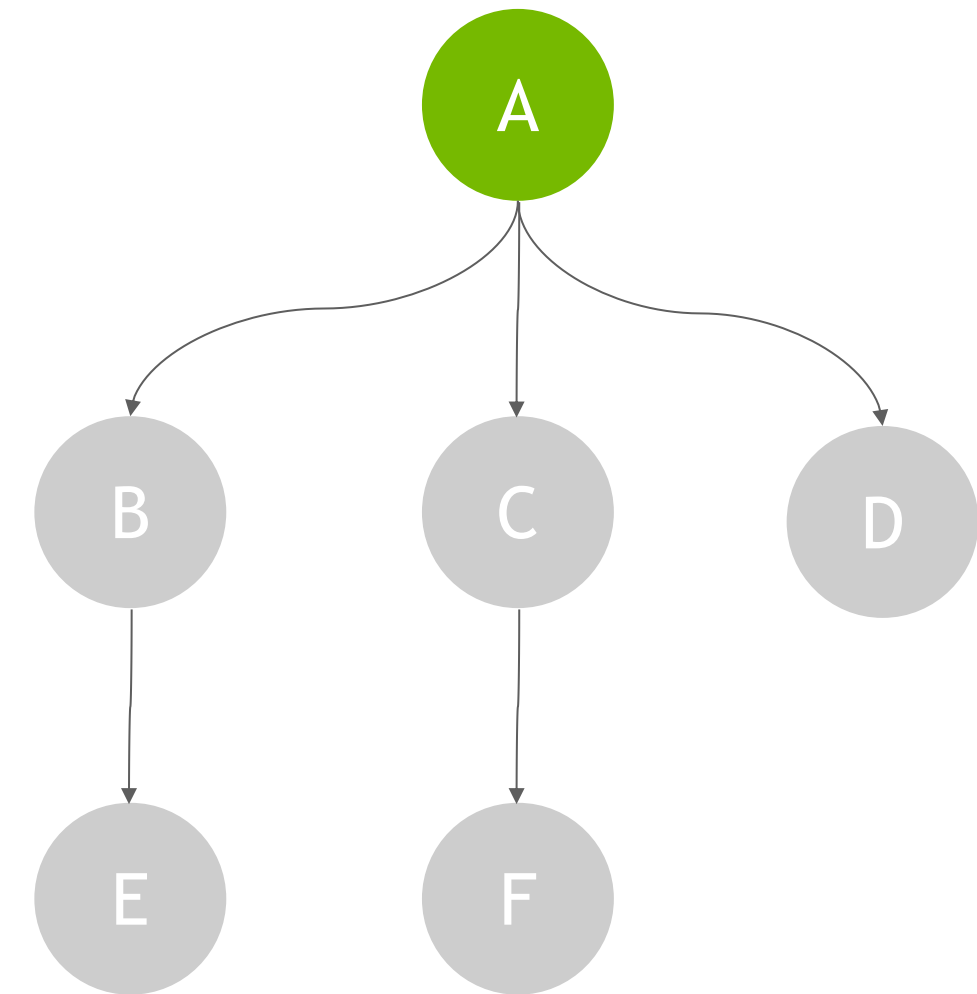


WHAT HAPPENS IN SESSION.RUN

How Are Operations Scheduled

```
ready_nodes = [A, ]
```

```
ScheduleReady(read_nodes)
```



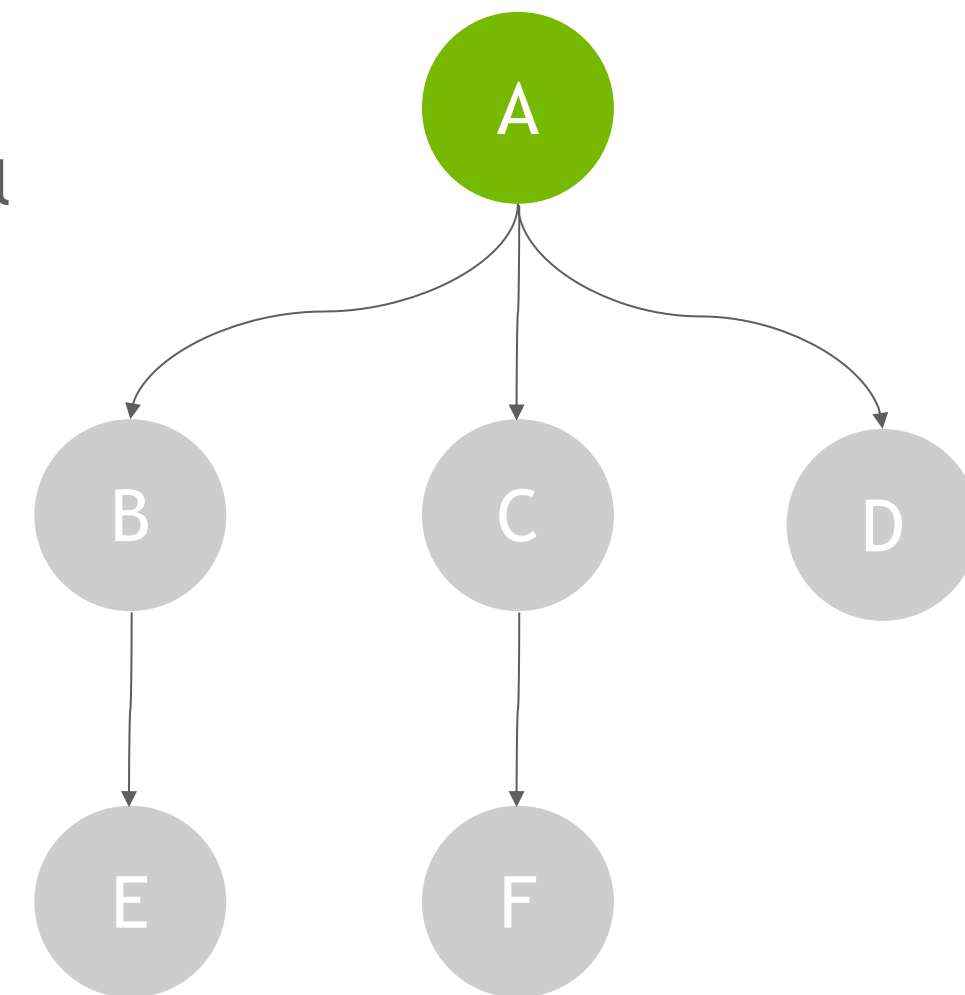
WHAT HAPPENS IN SESSION.RUN

How Are Operations Scheduled

ScheduleReady(read_nodes): # [A,]

for node in read_nodes: # [A,]

Process (node) # A, in a new thread got from thread-pool



WHAT HAPPENS IN SESSION.RUN

How Are Operations Scheduled

Process(node): # A

inline_ready = [node,] # A,

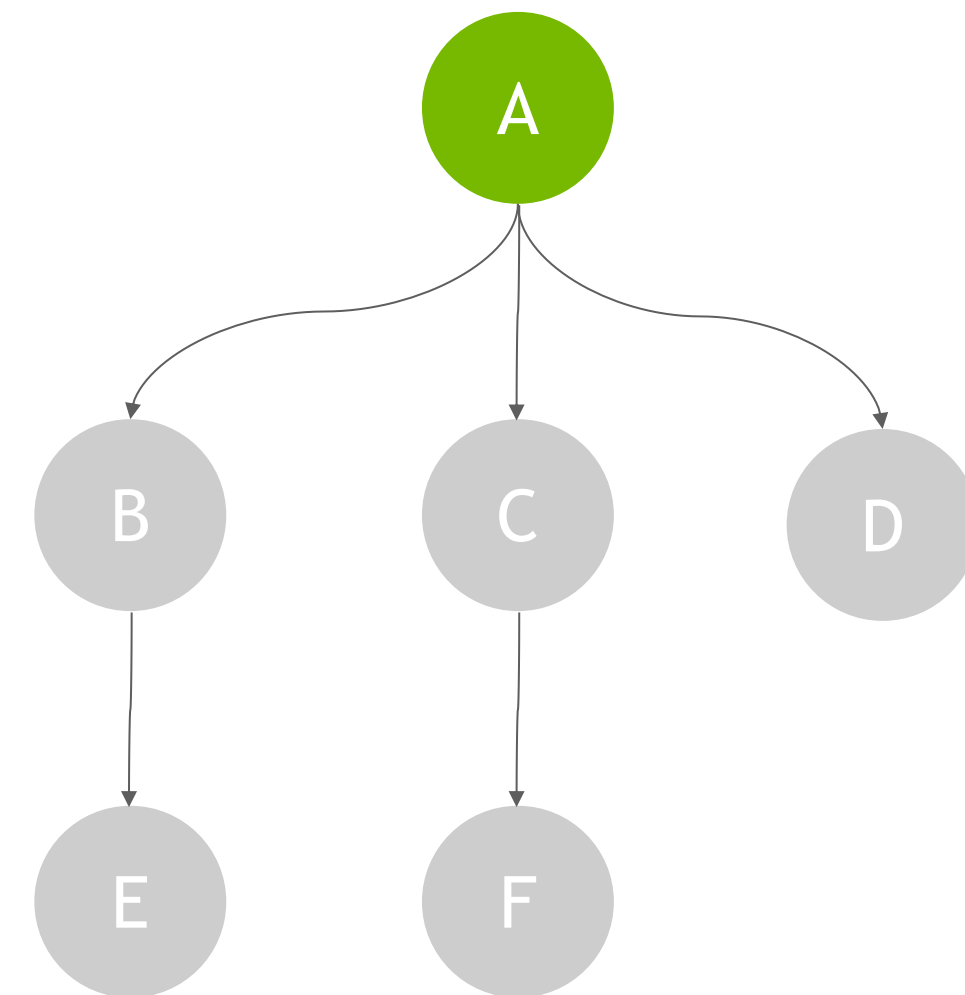
while inline_ready no empty:

node = pop(inline_ready)

device.Compute(op_kernel, ctx) *

ready_nodes = [B, C, D]

ScheduleReady([B, C, D], inline_ready)



* The process of OpKernal (sync op)

WHAT HAPPENS IN SESSION.RUN

How Are Operations Scheduled

```
ScheduleReady(read_nodes, inline_ready): # [B, C, D], thread 2
```

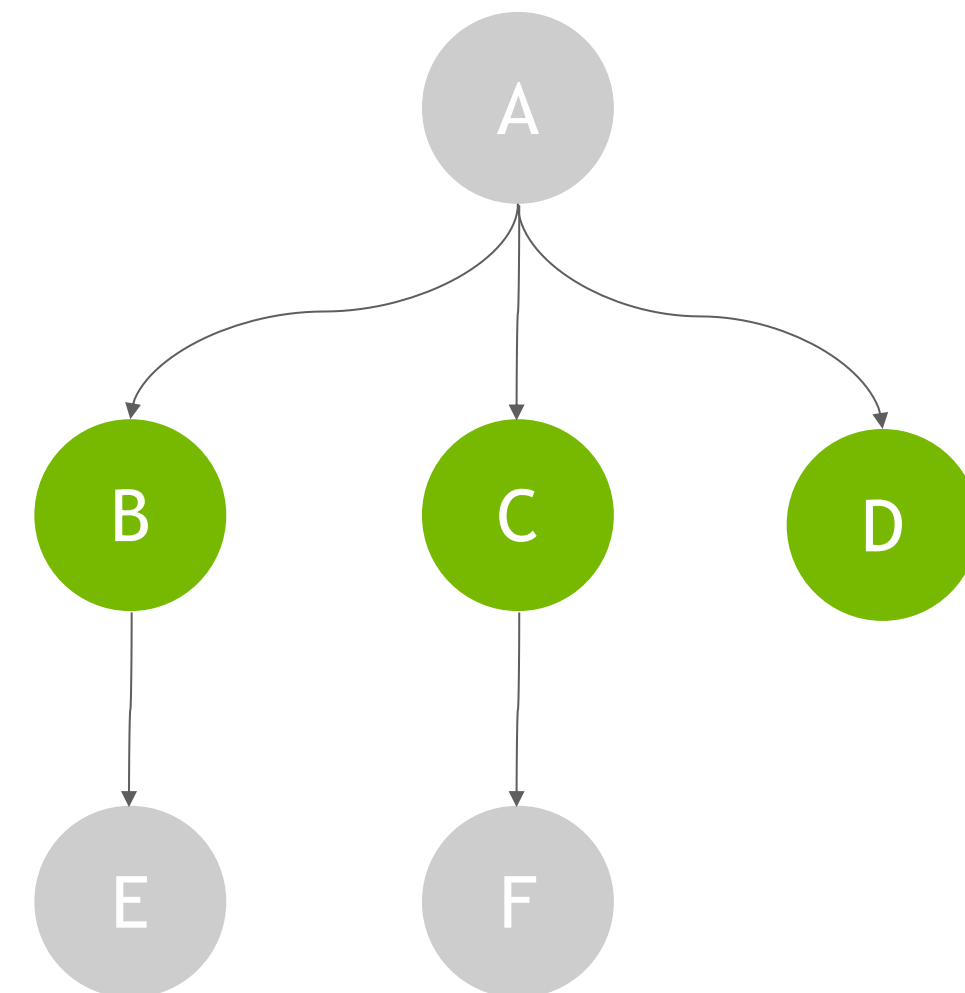
```
    for node in ready_nodes: # [B, C, D], thread2
```

```
        if node is expensive:
```

```
            Process (node) # thread 3,4,...
```

```
        else:
```

```
            inline_ready.push_back(node)
```



WHAT HAPPENS IN SESSION.RUN

How Operations Are Scheduled

Expensive



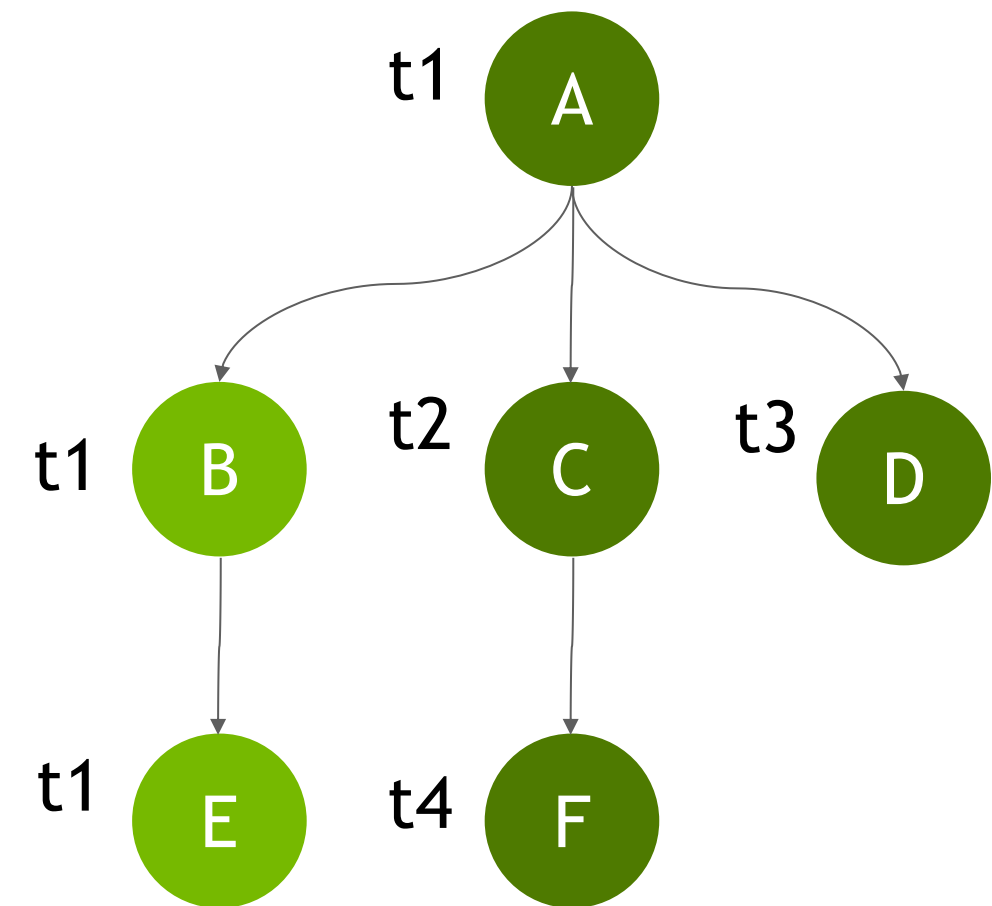
Inexpensive



Most CPU ops are expensive

GPU ops are inexpensive

OpKernels executing on GPU tie very few resources on the CPU where the scheduler runs

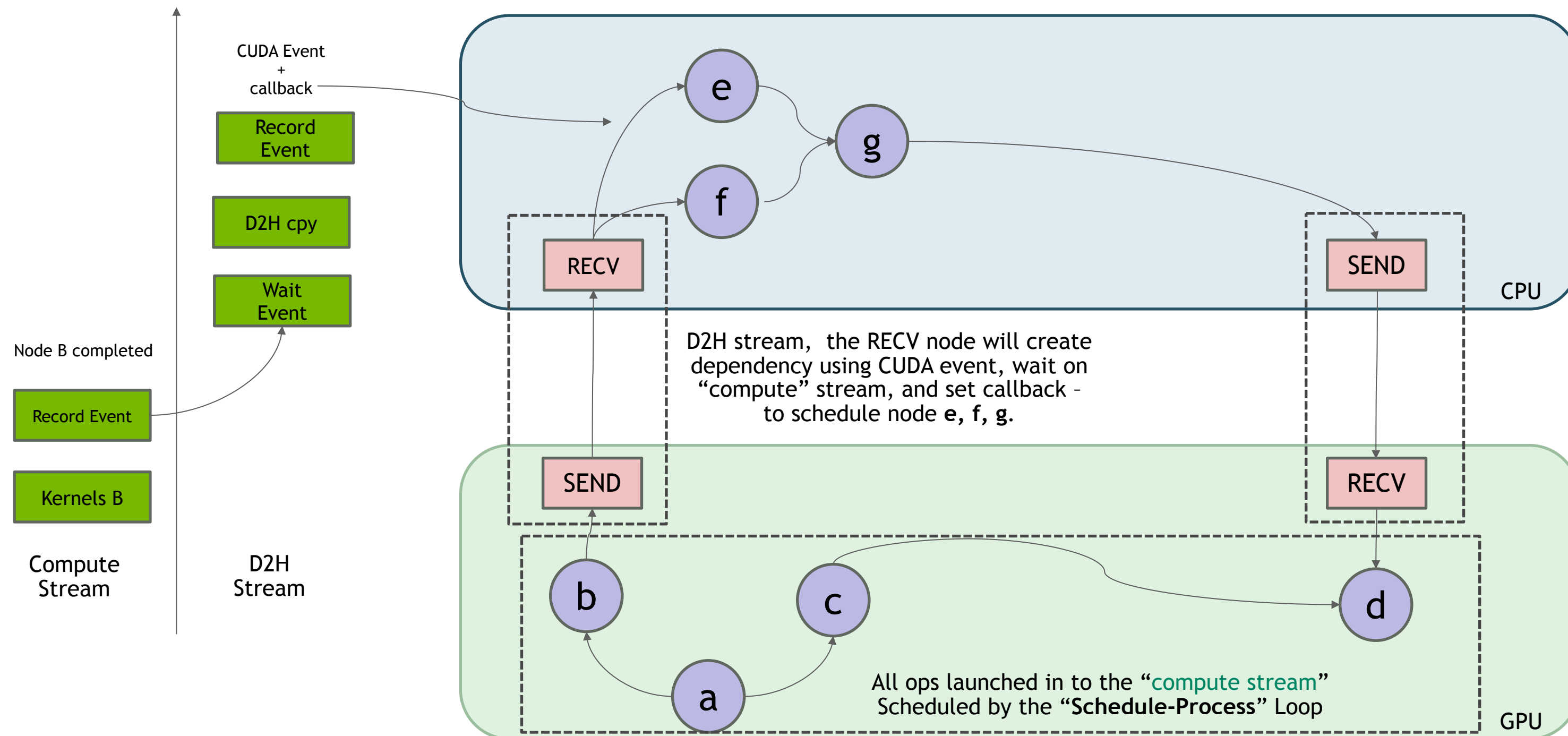


❑ Multiple threads launching could significantly increase the launch overhead

❑ If multiple threads are executing session run, the overhead will be worse

How TF Use CUDA Streams

A Group of Streams (1 for compute, 1 for h2d, 1 for d2h, several for d2d)



WHAT HAPPENS IN SESSION.RUN

CUDA Events Management

CUDA Events are used to create dependencies between streams

TF reuses events if possible

Maintain two lists of events:

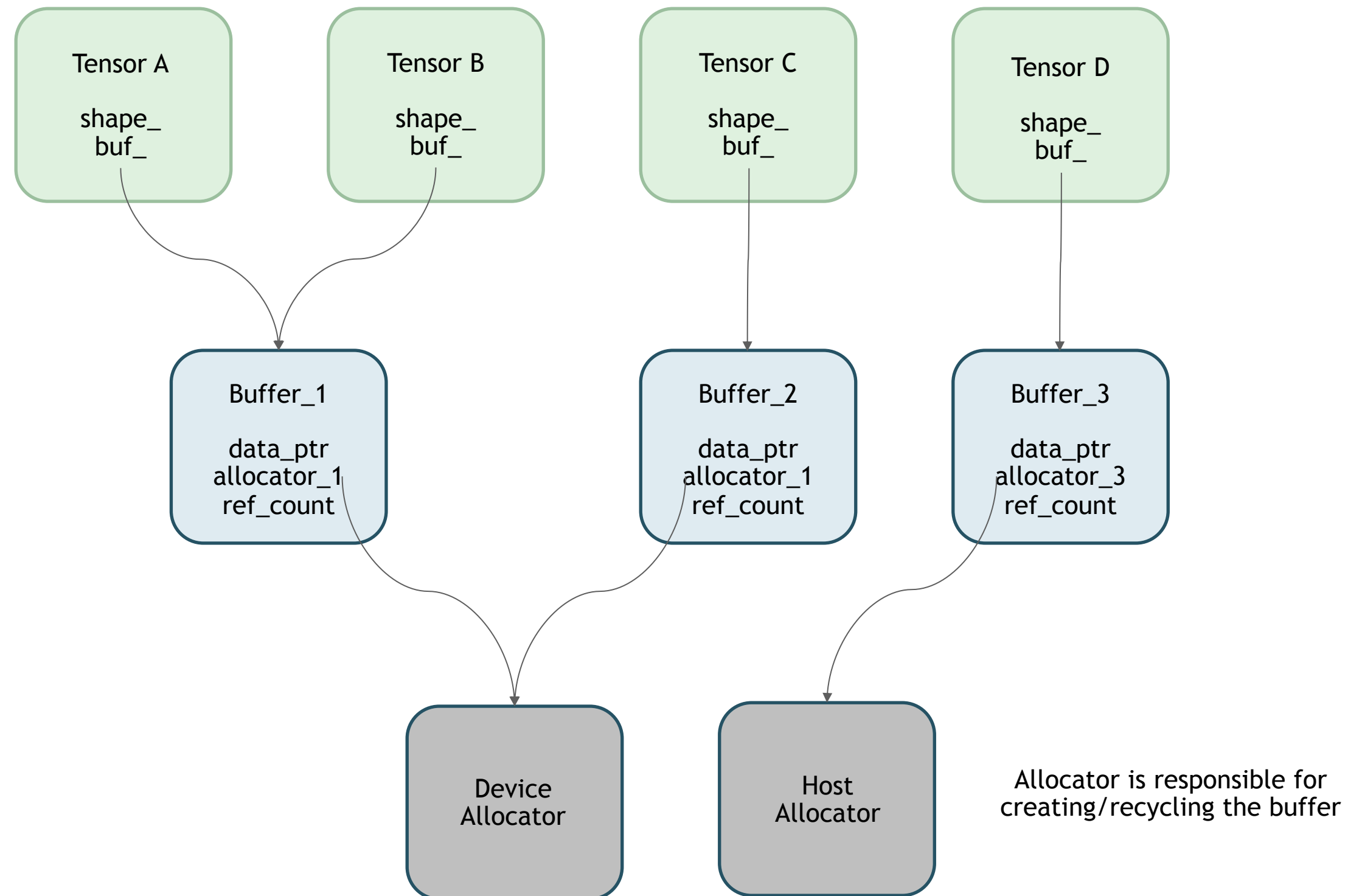
- Free Events
- In-use Events

When need to record an Event, get one from free event list, or create a new one (append to in-use lists)

There is a thread periodically check the in-use event list, move completed events to free event list

WHAT HAPPENS IN SESSION.RUN

How does TF Manage Memory



CONTENT

What happens in TF session run

Integrate CUDA Graph into TensorFlow

Use case: Alibaba Search Recommendation System

INTEGRATE CUDA GRAPH INTO TENSORFLOW

Overview

Use only 1 stream for Compute, H2D, D2H, D2D

Simplify the capture

Disable syncs during session runs

Syncs are not necessary, as in capture mode, all GPU operations are not executed but just recorded

Event queries are also disabled

Memory management

Hold the Tensors allocated during capturing

Tensor reusing

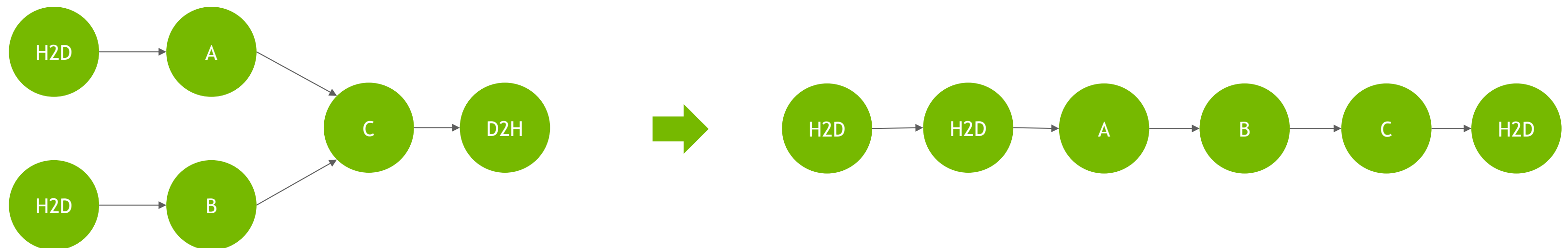
INTEGRATE CUDA GRAPH INTO TENSORFLOW

One Stream Executing

Use only 1 stream for Compute, H2D, D2H, D2D

Simplify the capture process, no need to use CUDA Events to build the dependencies

Pitfall: The original Graph flattens into a straight line, no parallelism between memcopy nodes and computing nodes (this issue can be solved by using multiple-graphs and multiple-streams)

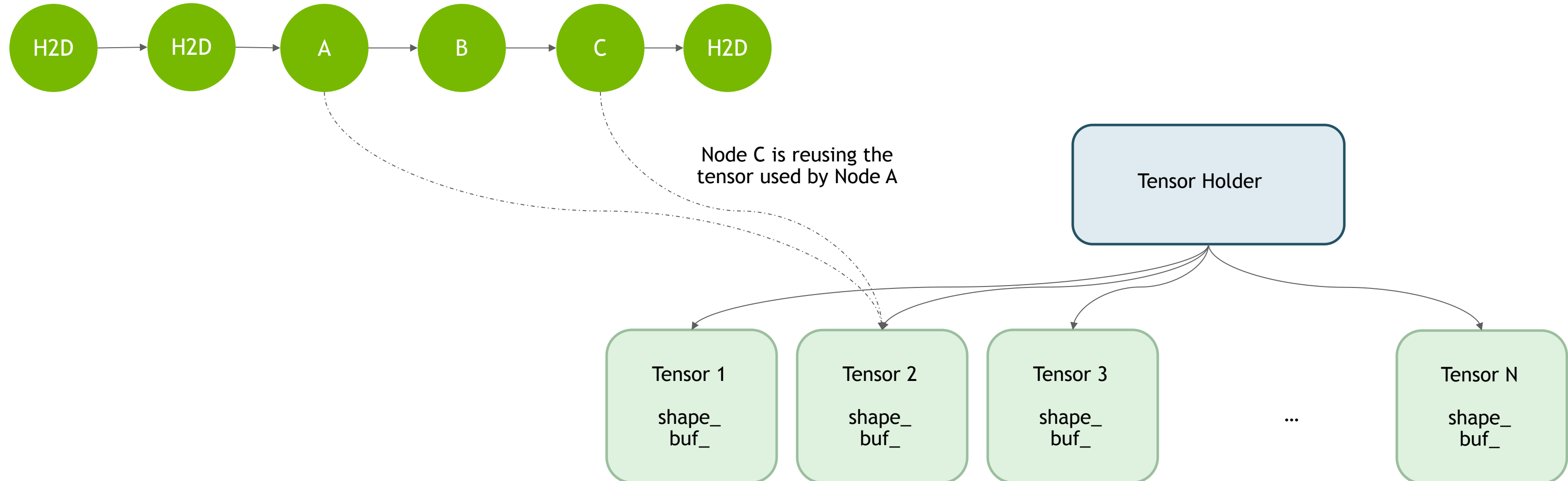


INTEGRATE CUDA GRAPH INTO TENSORFLOW

Memory Management

Each Graph has a “Tensor Holder” object which holds the tensors (allocated during session run), so memory for subsequent CUDA Graph launch and for normal session runs are isolated

Tensor Holder is also responsible for reducing memory footprint (by reusing tensors *)



INTEGRATE CUDA GRAPH INTO TENSORFLOW

Post Capture Process

After capturing, post process the CUDA Graph

☐ Validation of H2D/D2H nodes

- ☐ Condition 1 : Each H2D node in the graph, is either corresponding to an input tensor or const CPU tensor which is held by Tensor Holder
- ☐ Condition 2: Each D2H node in the graph, is corresponding to an output tensor
- ☐ If either condition 1 or condition 2 fails, the graph is invalid (there may be uncaptured CPU operations)

☐ H2D nodes removal

- ☐ Remove the H2D nodes corresponding to input tensors, and record the (host_src --> gpu_dst) mappings for the input tensors
- ☐ Eliminate the needs to do extra H2H (host to host) copies before launching the CUDA Graph, and user is responsible for the H2D copies of input data based on the (host_src --> gpu_dst) mapping

☐ Instantiate the CUDA Graph

- ☐ Create the Executable CUDA Graph instance

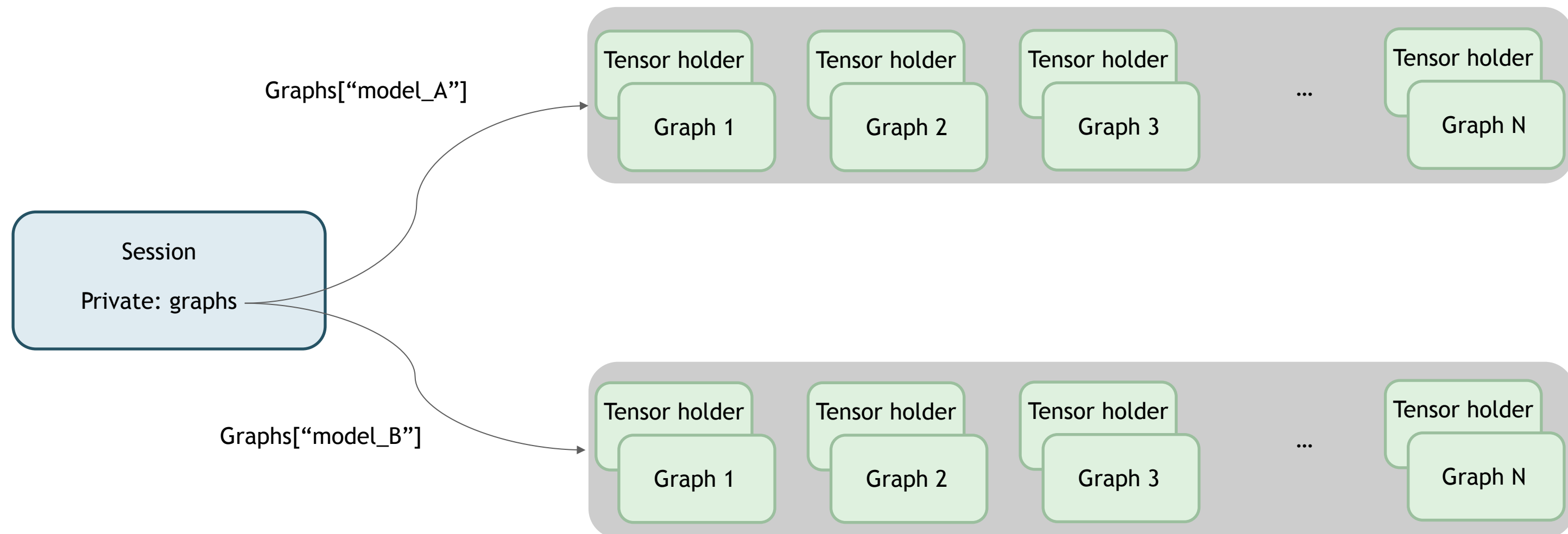
INTEGRATE CUDA GRAPH INTO TENSORFLOW

Graph Management

Graphs and Executable Graph Instances (and the corresponding tensor holders) are held by Direct Session object

Get a Graph by specifying the model's name and the graph index

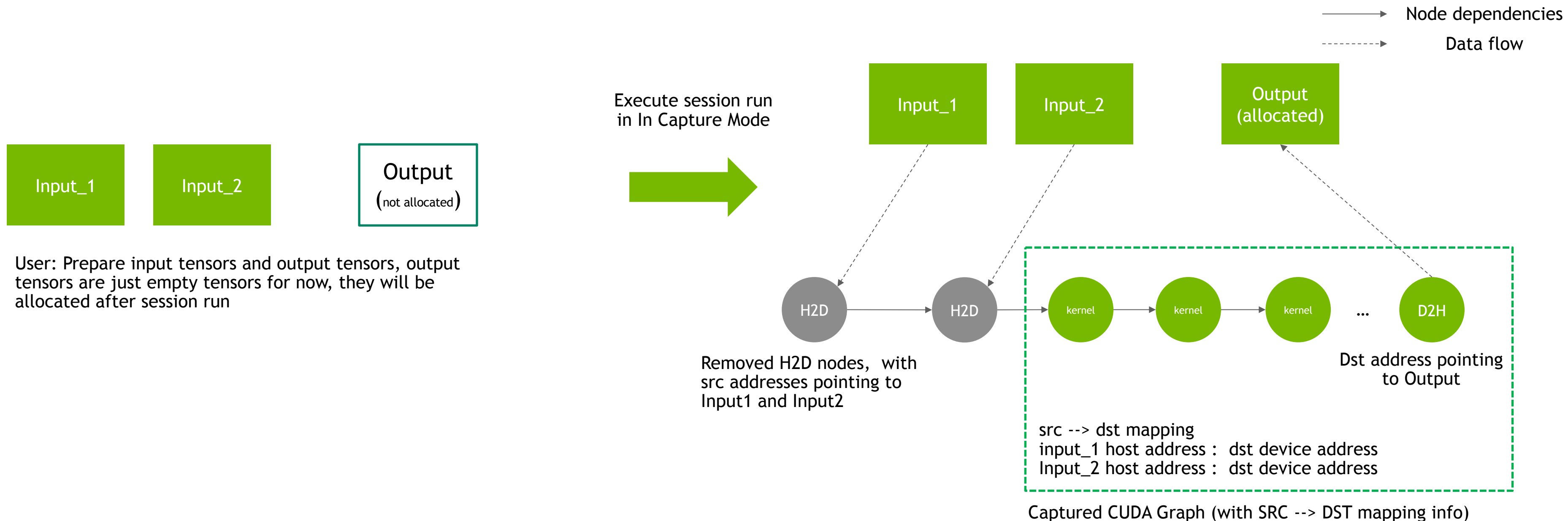
`DirectSession::DestroyCudaGraphs()` will destroy the CUDA Graphs and corresponding tensor holders (release memory)



INTEGRATE CUDA GRAPH INTO TENSORFLOW

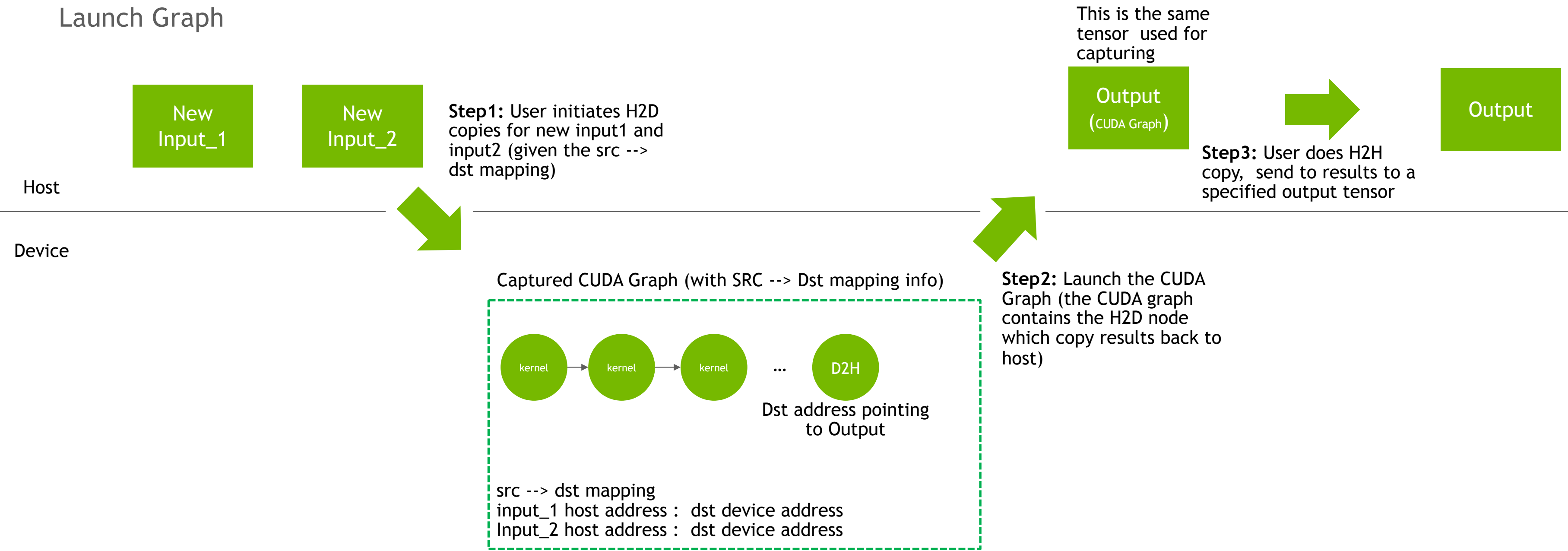
Workflow

Capture Graph



INTEGRATE CUDA GRAPH INTO TENSORFLOW

Workflow



CONTENT

What happens in TF session run

Integrate CUDA Graph into TensorFlow

Use case: Alibaba Search Recommendation System

USE CASE: ALIBABA SEARCH RECOMMENDATION SYSTEM

Recommendation Network

Sparse Part

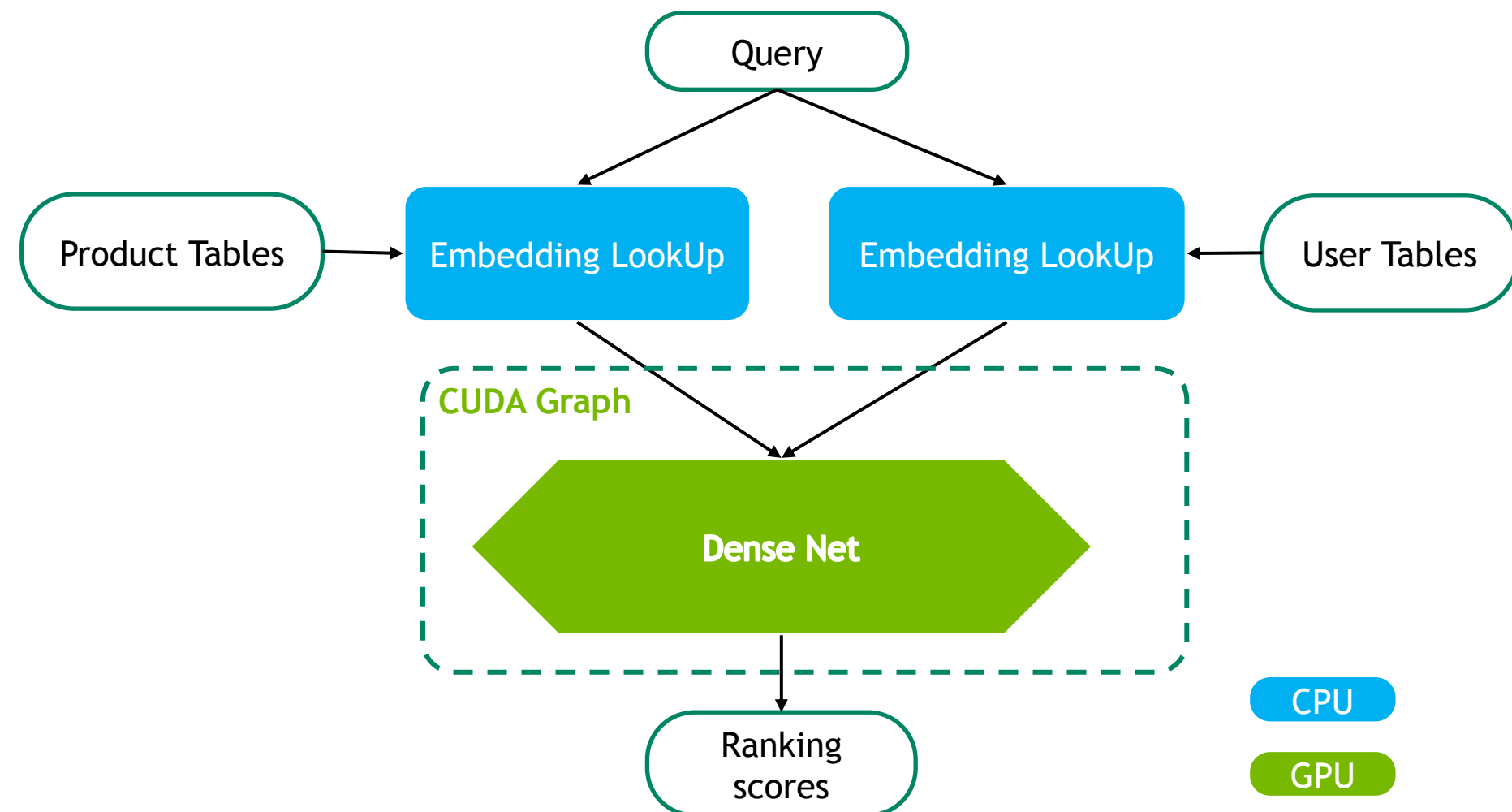
- Embedding lookup

Dense Part

- MLP or MultiHeadAttention
- Consist of dense computations
- Suffer op launch overhead problem

How:

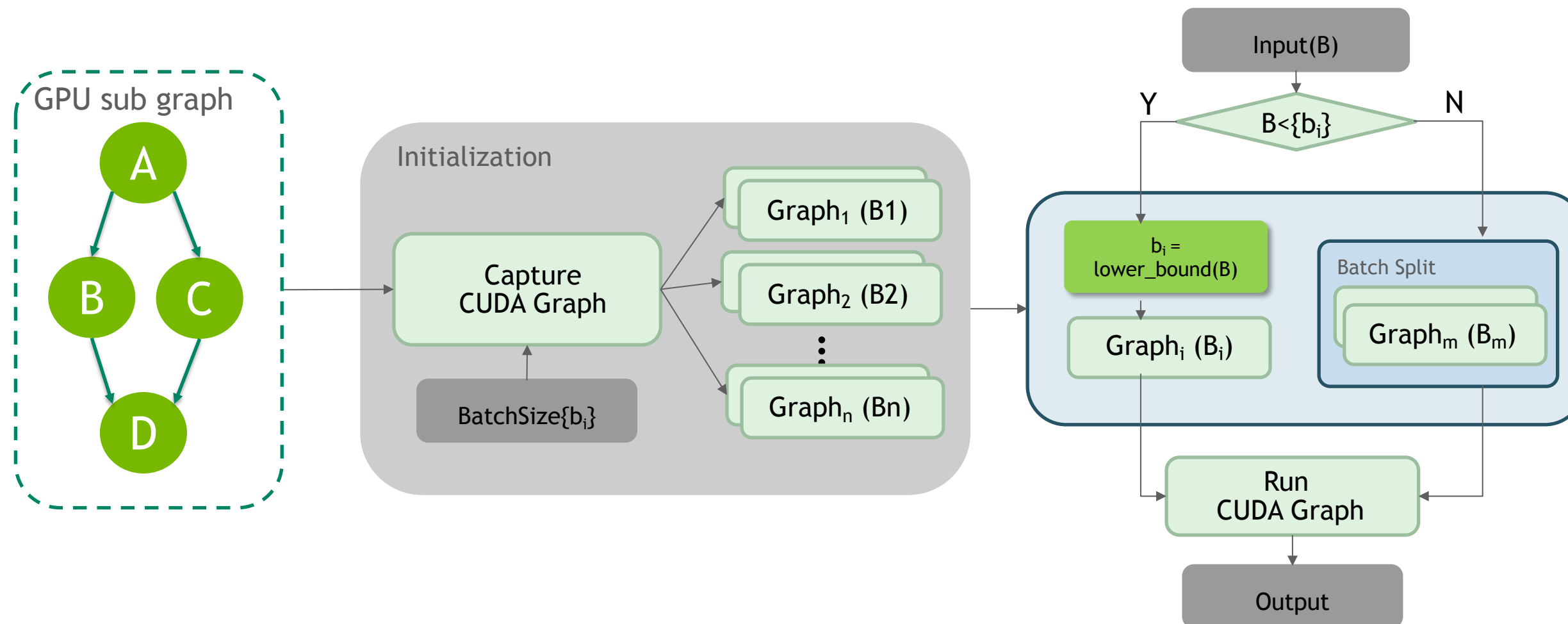
- ☐ Place sparse part on CPU
- ☐ Place dense part on GPU
- ☐ Optimize dense part with CUDA Graph



USE CASE: ALIBABA SEARCH RECOMMENDATION SYSTEM

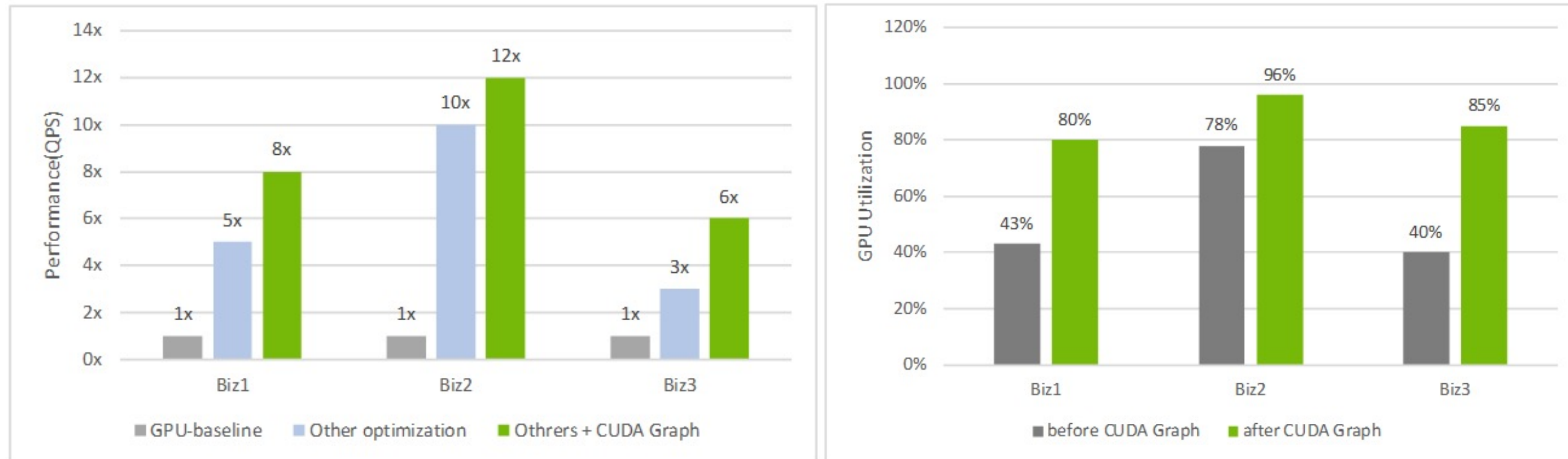
Workflow

- **Divide model:** Divide model into CPU sub graph and GPU sub graph (CUDA Graph)
- **Initialization (Offline model):** Capture all GPU activities and stored as graph in GPU memory (N batchsize, M streams $\rightarrow$ M * N graphs)
- **Execution (Online model):** Select graph and run in CUDA Graph model

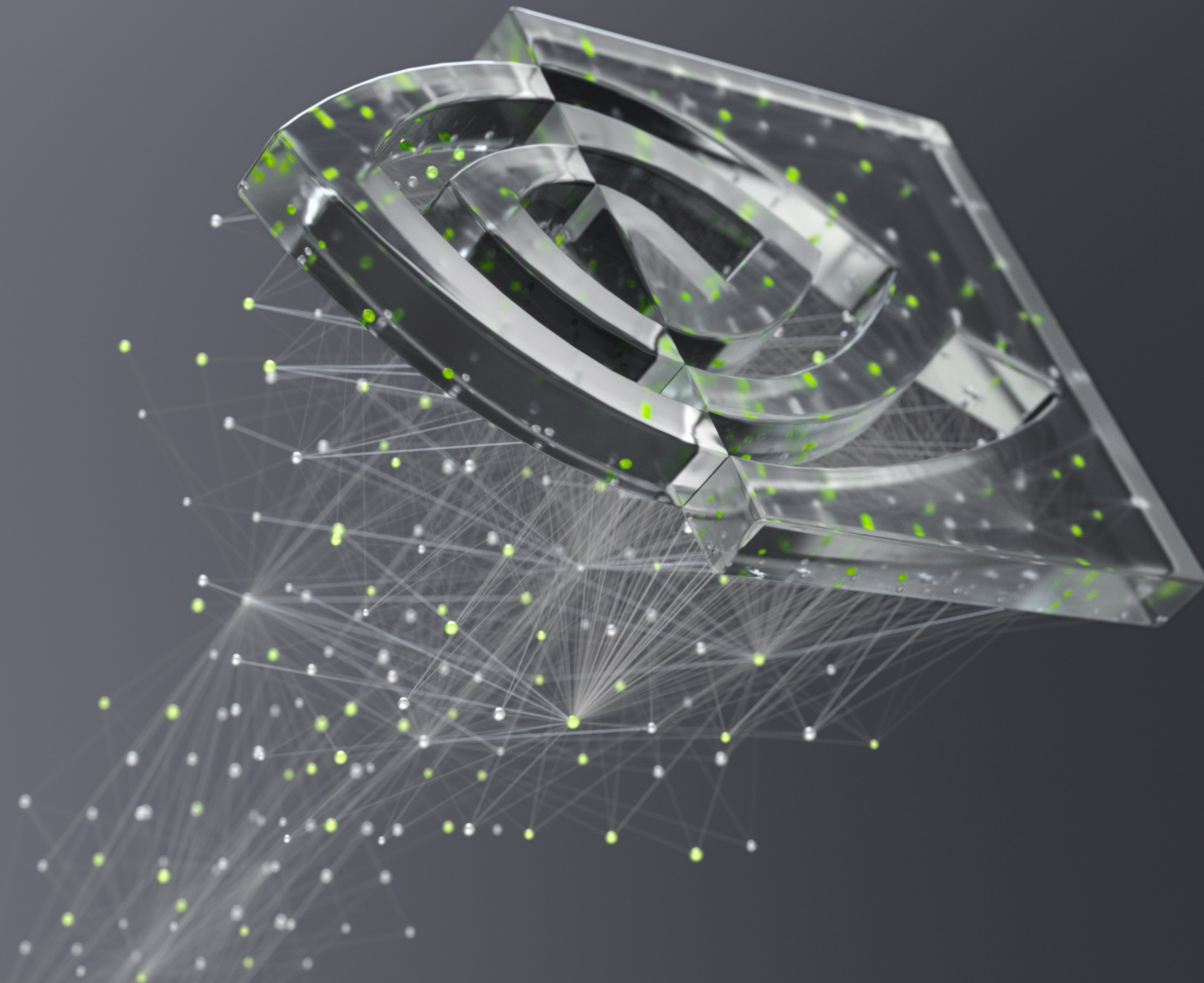


USE CASE: ALIBABA SEARCH RECOMMENDATION SYSTEM

Experimental Results



- QPS(Query per second): 50% improvement on average
- Rt(Response time): 10% reduction on average
- GPU utilization : $\geq 80\%$



 NVIDIA
GTC

